

Apigee Notes

25/07/26

Apigee Interview (Apigee / Apigee X)

JD (Role)

- Design, Develop, Deploy & Support
- Configure
- API proxies
- API Products
- Developer Portals
- Security Policies
- Monitor API Performance
- Troubleshoot production Issues
- Root cause Analysis.
- Work with development teams to integrate backend services with API Gateway.
- Implement API security using.
- OAuth 2.0
- JWT
- API keys (VerifyAPIKey)
- SSL / TLS.
- Support API lifecycle management, Versioning & deployment activities.
- Collaborate with cross-functional teams to deliver scalable & secure API solutions.
- Participate in production Support, issue resolution & performance tuning.
- Skills
- GCP
- CI/CD pipelines.
- Docker
- Kubernetes
- JSON / XML
- HTTP / HTTPS.
- Postman
- Git

- Shell scripting
- Monitoring.

API Policies

- Security policies → (OAuth, JWT, APIkey & Basic Authentication).
- Rate limiting policies → (Spike Arrest, Quota & Cache policies).
- Response Cache
- Lookup Cache.
- Populate Cache

Need to be clear

- Parameters (Query, form)
- Headers
- request body (data). (Format).
- To understand the Flow
- What is going forwarding to the request (when user hit the endpoint).
- Based on the request
- How your API proxy is going to react (validating the user).
- How you are going to Validate (what policies you have added to your proxy to validate the user).
- You need to explain the flow when request comes in & how your API proxy will react (Validate → steps).

Basic Authentication Policy

- User Send the Encoded token in the Authorization header.
- In Apigee → My proxy → I have added Basic Authentication policy & **KVM** (Key Value Map) policies.
- 1st → I will Get (retrieve) the values from **KVM** in my **KVM** entries like username & password.
- 2nd → In my Basic Authentication policy → I will decode the Basic auth key (client sent in the Auth header).
- After decoded → using **KVM** values which i retrieved. (Validate using the **KVM** values).
- If it's succeed (Valid) → request send to Backend (Target URL) → 200 resp.
- Else getting 401 Unauthorized Error.

Verify API key Policy

- Client → send the API key in the header

- Apigee → API proxy will Validate the API key by using **VerifyAPIKey** policy.
- Create API proxy & add **VerifyAPIKey** policy & Deploy the proxy.
- Create the API product & add the API proxy.
- Create the App & add the API product.
- In the App → have client ID & client secret (key & secret)
- Take the client ID & Use the client ID as a header in the API request
- In the API proxy flow will **VA** -policy → Validate the key & forward the request once key is valid.

[SSL → Secure Sockets Layer]. [TLS → Transport Layer Security].

SSL/TLS

- HTTP → Unsecure connection
- default port → 80.
- HTTPS → secure connection
- S → stands for certificate
- port → 443.
- TLS → One Way TLS
- client Validates server certificate only.
- mTLS → Two Way TLS
- Both client & Server Validates certificates.

TLS in Apigee Edge

- Northbound TLS (client → Apigee)



- Encrypt traffic b/n API consumers & Apigee.
- Protect credentials, tokens & payloads.

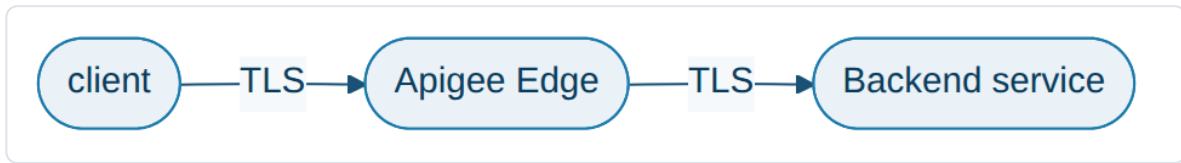
Configuration

- Virtual Host
- key store
- Trust store.

Eg <https://api.company.com/orders>

- client Validates Apigee's certificate before sending data.

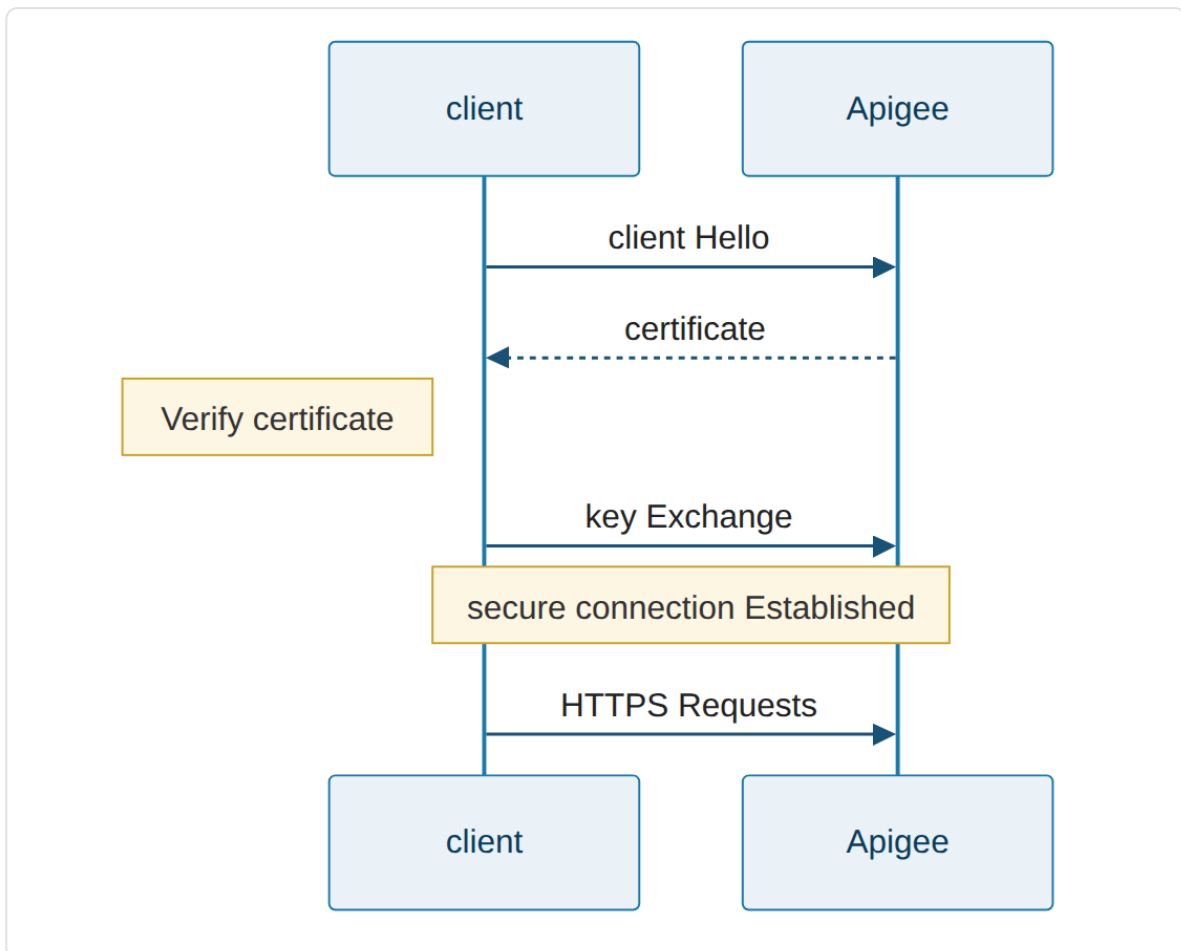
Southbound TLS (Apigee → Backend)



- Purpose → secure communication between Apigee & Target servers.

Eg Public API's using TLS

TLS Handshake Flow



Note: In TLS → client Verifies Server

- server does not verify client.

mTLS → (Mutual TLS)

- server provides Identity to client.

- client provides Identity to Server.
(Both sides exchange certificates).

mTLS Flow



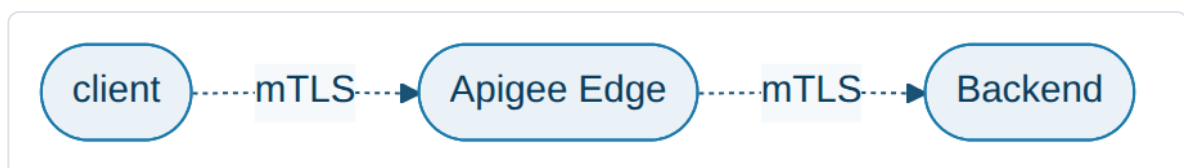
- client cert
 - server cert.
- (Both parties authenticate each other).



Eg

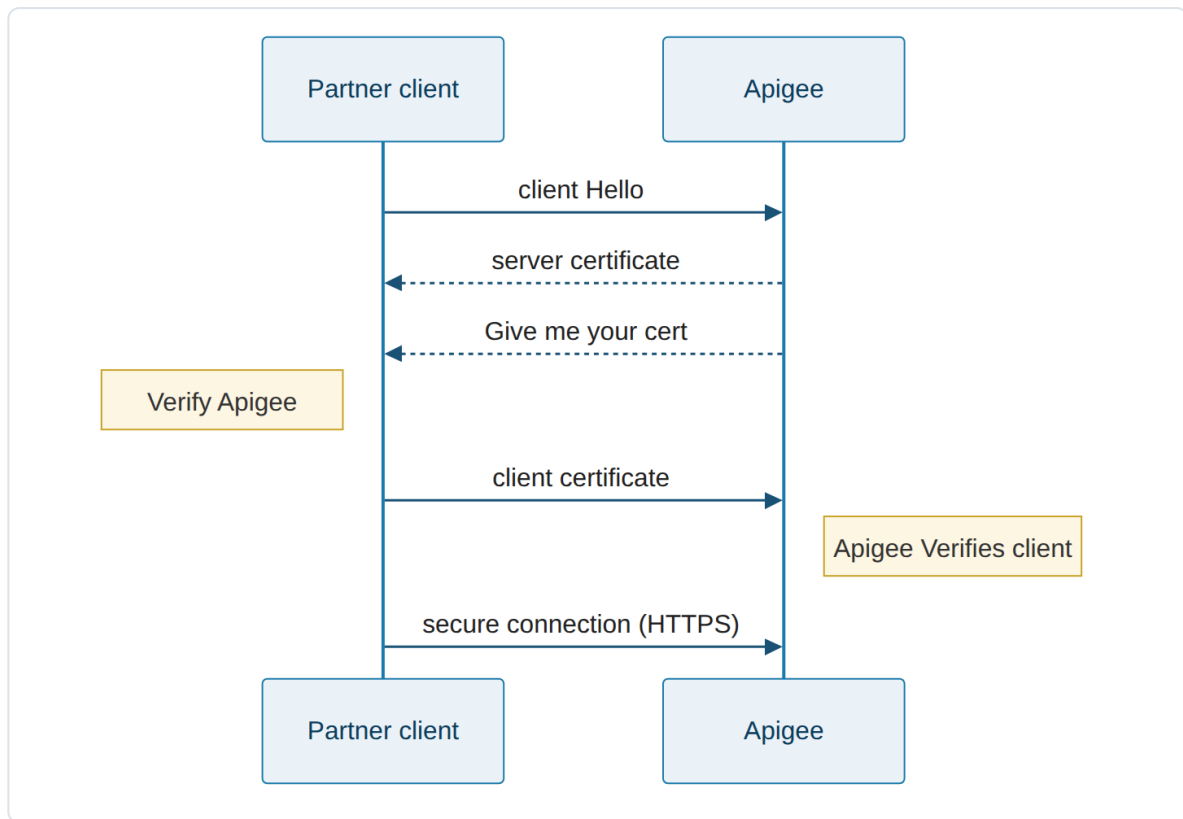
- Banking API's
 - Healthcare API's
 - B2B integrations
- client certificate is mandatory.

mTLS on Southbound connection



- Backend requires Apigee to present a client certificate.

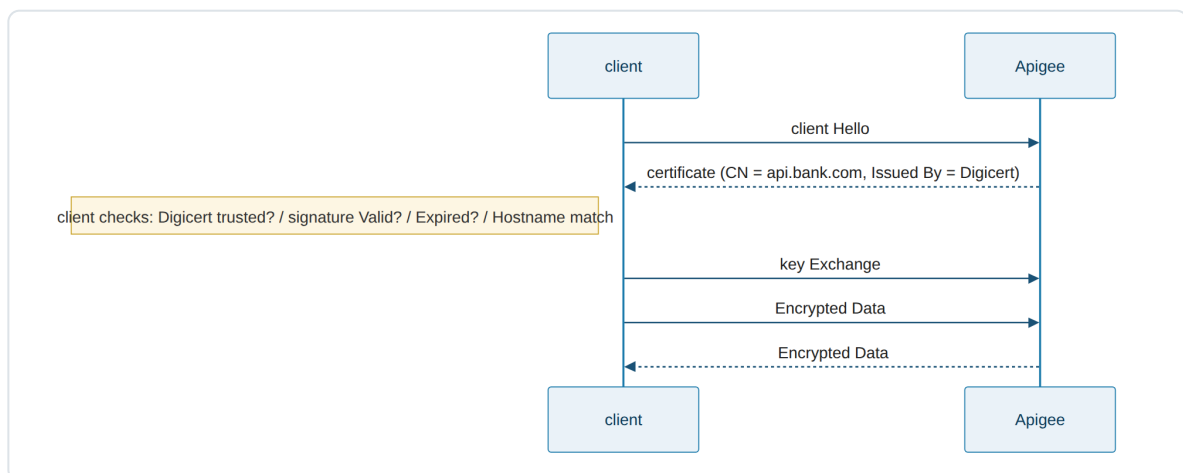
mTLS Flow (Handshake Flow)



Key store

- Keys are stored in Apigee key store.

Complete Flow:



Virtual Host: (Routing)

- Virtual Host tell Apigee
- which domain name to listen

- which port to use (80, 443).
- whether Traffic is HTTP (or) HTTPS.
- which SSL certificate should be used for HTTPS.

Eg:

- Name: secure-vhost
- Host Alias: api.company.com
- Port: 443
- Protocol: HTTPS.

⇒ <https://api.company.com/customer>

- Apigee Router receives the request and matches the Host header (api.company.com) with the configured virtual host.

Eg → Virtual hosts: <https://api.company.com> (instead of) <https://org-test.apigee.net>

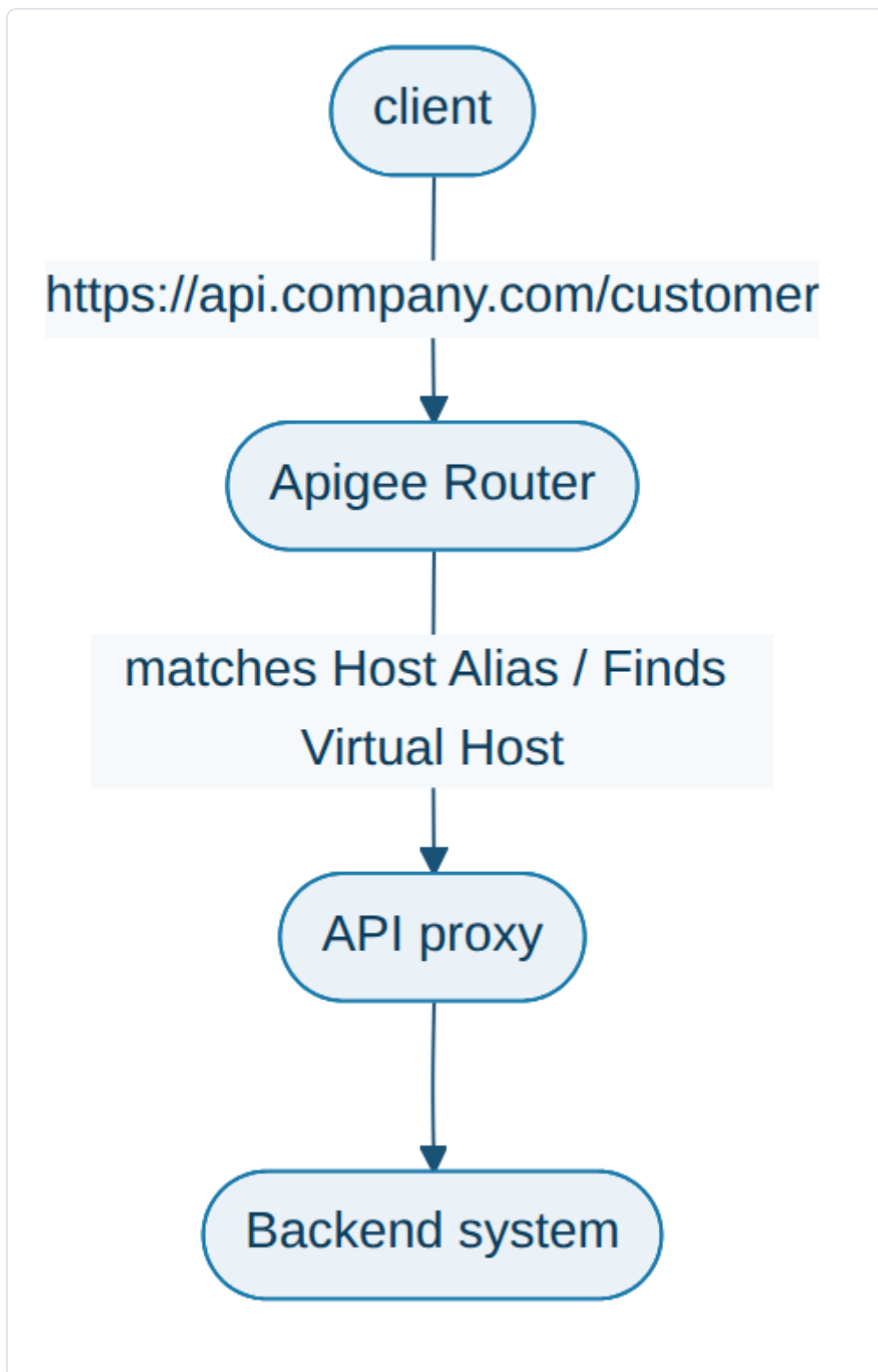
→ HTTP

- default
- Port : 80

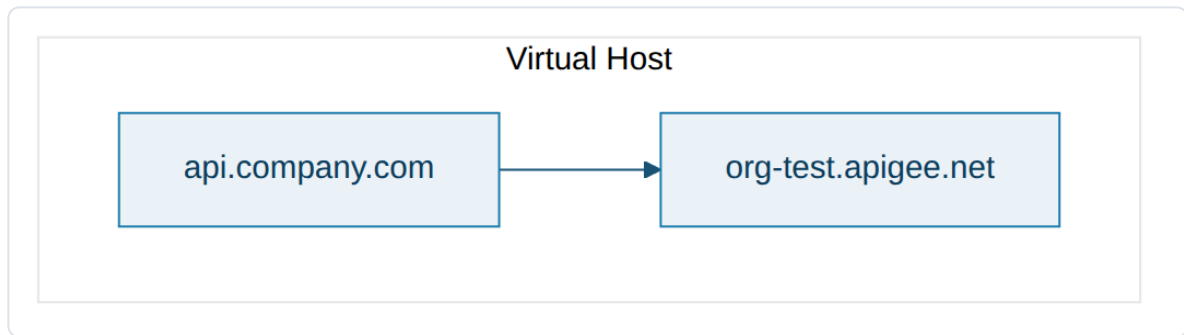
HTTPS

- secure
- Port : 443.

Flow Diagram



Eg: (Virtual Host)



Here → (Virtual Host)

- api.company.com → Network entry point.
- /customer → proxy routing path.

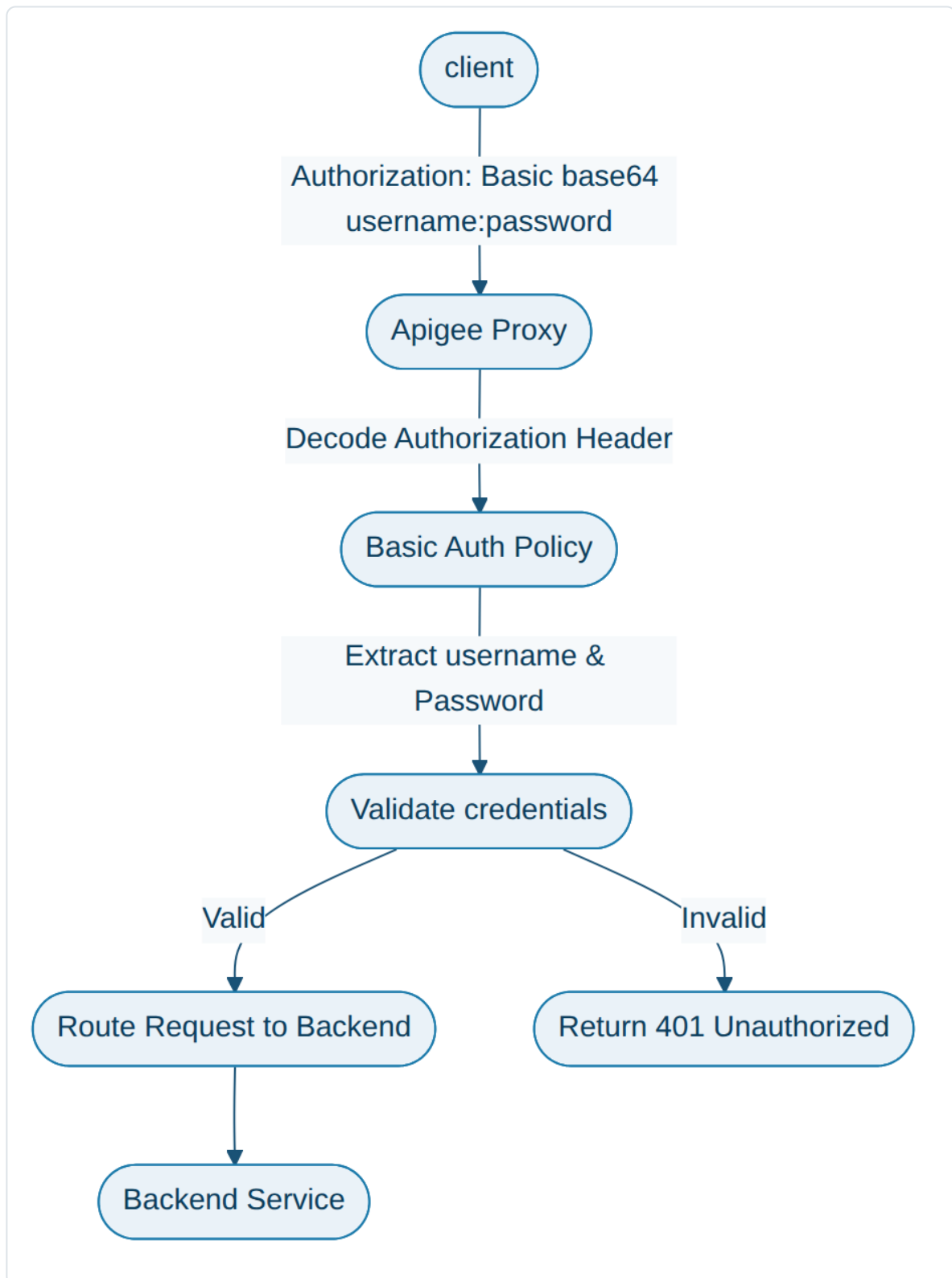
Note:

- The Router uses the request's Host header to determine which virtual host should process the request.

⇒ Instead of giving Apigee default URL (As per org standard) → create one Virtual Host (api.company.com) & give to client

26/07/26

Basic Authentication Flow



Real Time Example

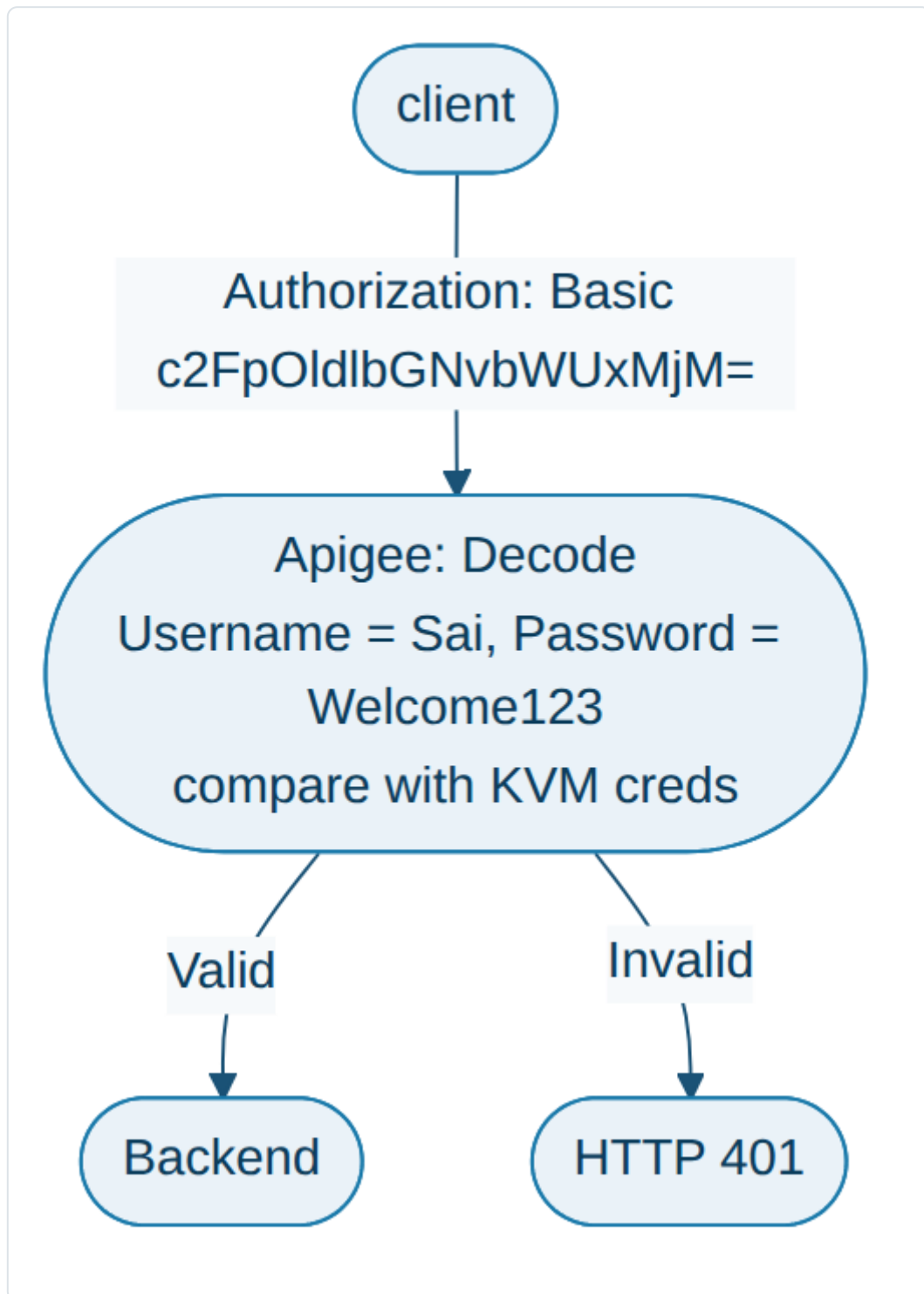
Username : Sai

Password : Welcome123

client encodes → Sai:Welcome123

Base64 → c2FpOldlbGNvbWUxMjM=

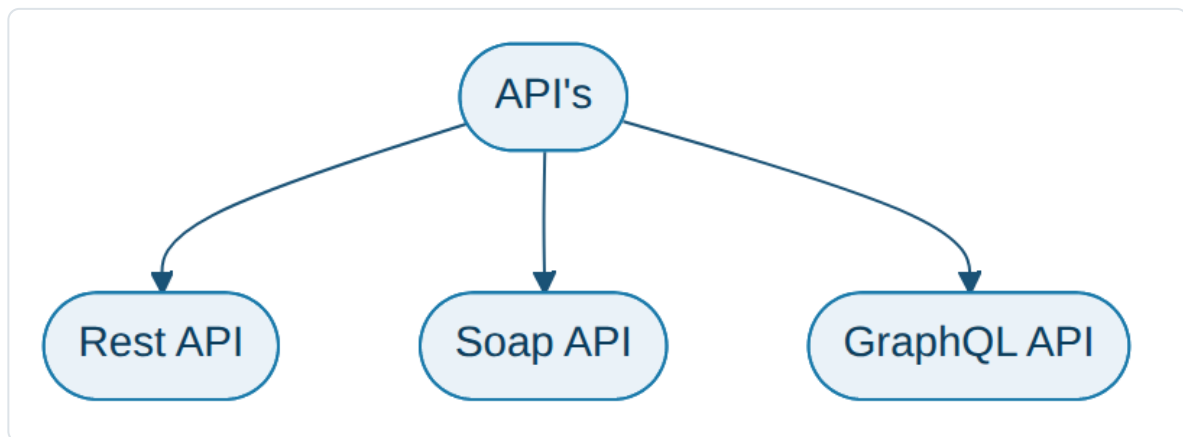
Flow



IBM API connect → Apigee X migration

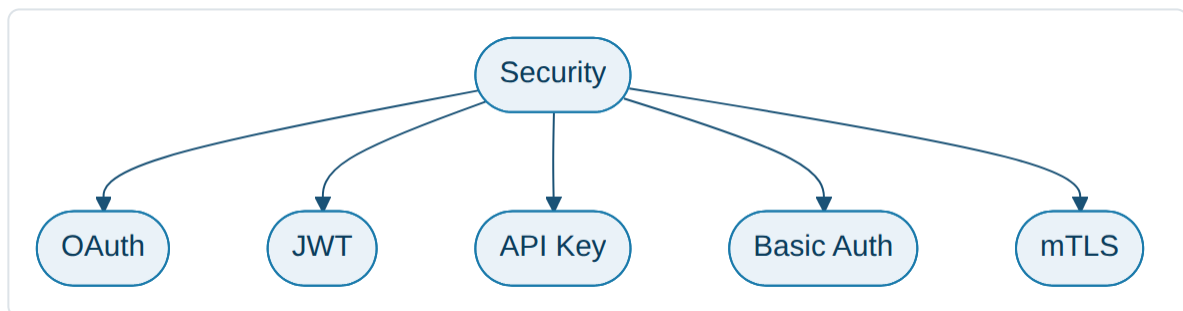
- Before migrating the API proxy from IBM API connect to Apigee X need to follow below checklist

1. Understand the API Pattern



- Based on the API pattern we need to create the API proxy.

2. Understand the security Mechanism (IBM API connect)



3. Understand the policies structures (IBM API connect)

- Transformation
- Routing
- Caching
- Rate limiting
- Logging
- Exception Handling

4. Backend systems

- HTTP / HTTPS
- SOAP

5.

- Once you understand all → create the spreadsheet to add the Information.
- API Name
- Version
- Target URL
- Policies
- Authentication

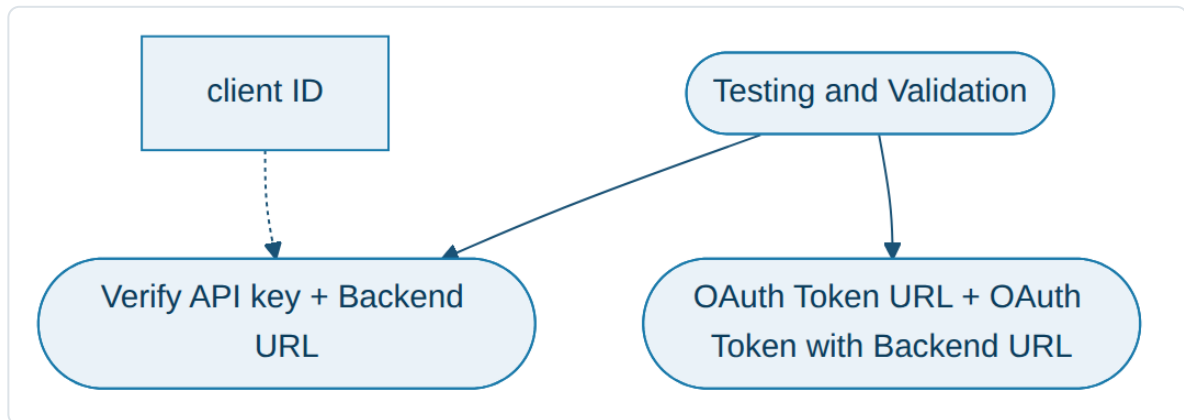
- Products

(Policies, Authentication, Products grouped with:)

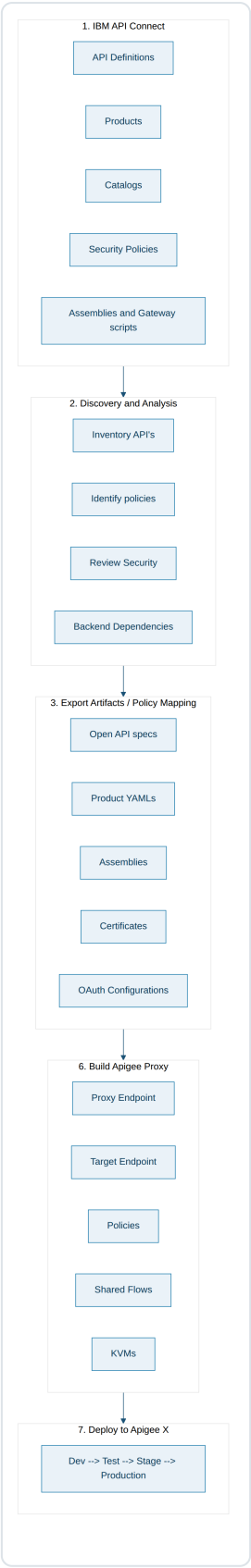
- Consumers
- Dependencies

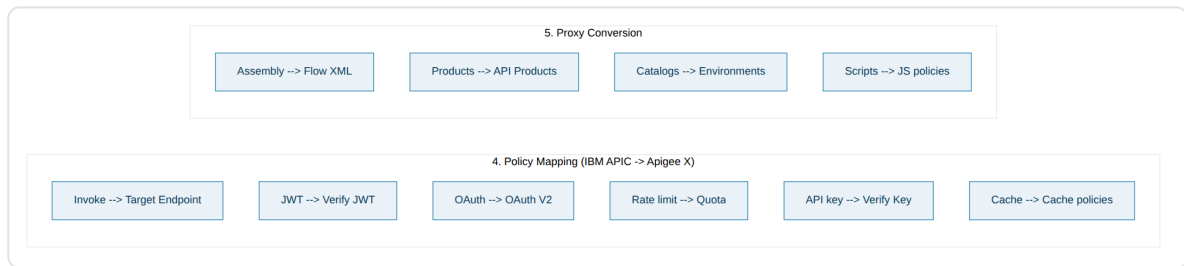
6.

- Once understand the requirement before creating the API Proxy in Apigee we need to do the Testing & Validation of IBM API connect Endpoint.



IBM API connect to Apigee X migration flow





once these are done

- Parallel run
- IBM API Connect
- Apigee X
- Cutover (IBM API Connect Decommissioning)

Simplified End to End Flow

Client Application → Apigee X → Verify API key / OAuth 2.0 / Verify JWT → Quota / Spike Arrest → Transformation → Target Endpoint → Backend Service → Response policies → Analytics & Logging → Client

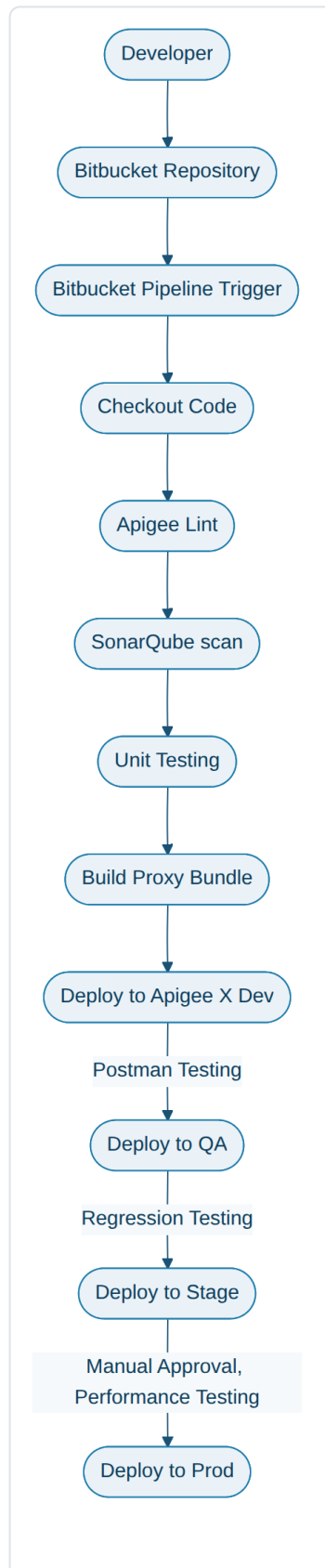
CI/CD Pipelines

- Bit Bucket
- Jenkins → Automation

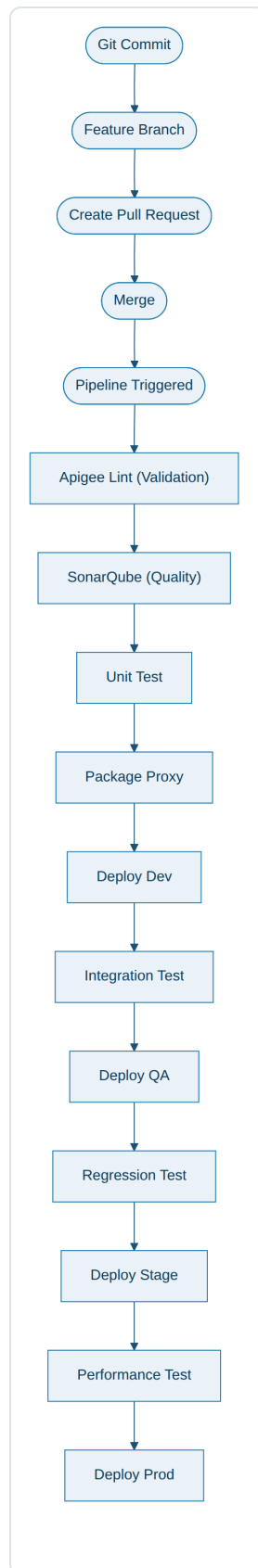
(Build & Deploy)

Apigee X Deployment Using Bitbucket Pipelines

Architecture flow:



Bitbucket Pipeline Flow

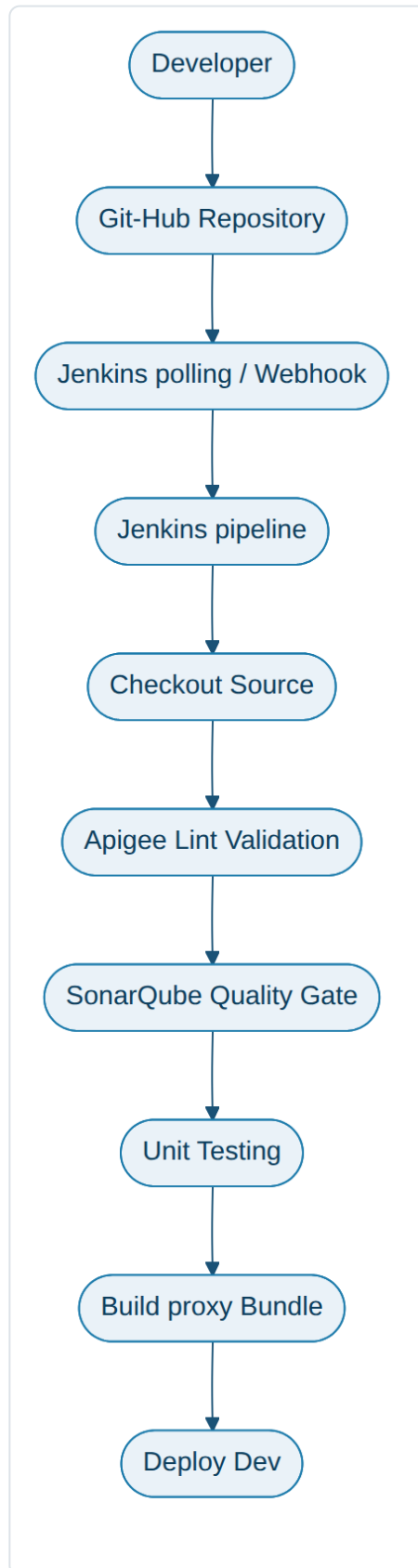


(For smaller workflows we use Bitbucket)

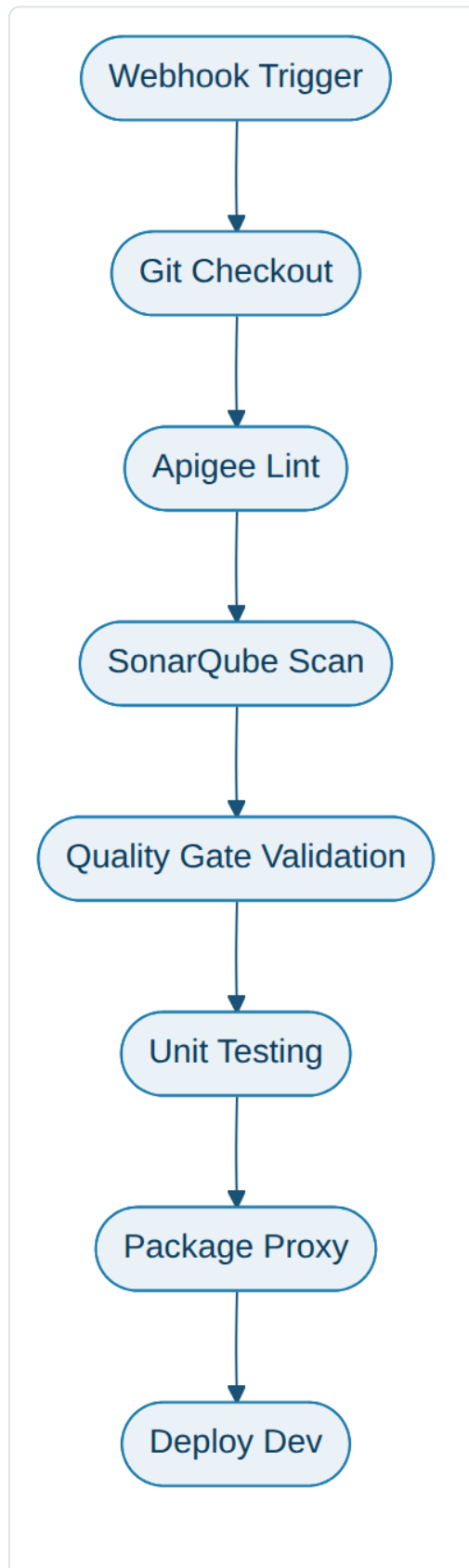
Deployment → Apache Maven plugin commands

2. Apigee X Deployment using Jenkins

Architecture Flow:



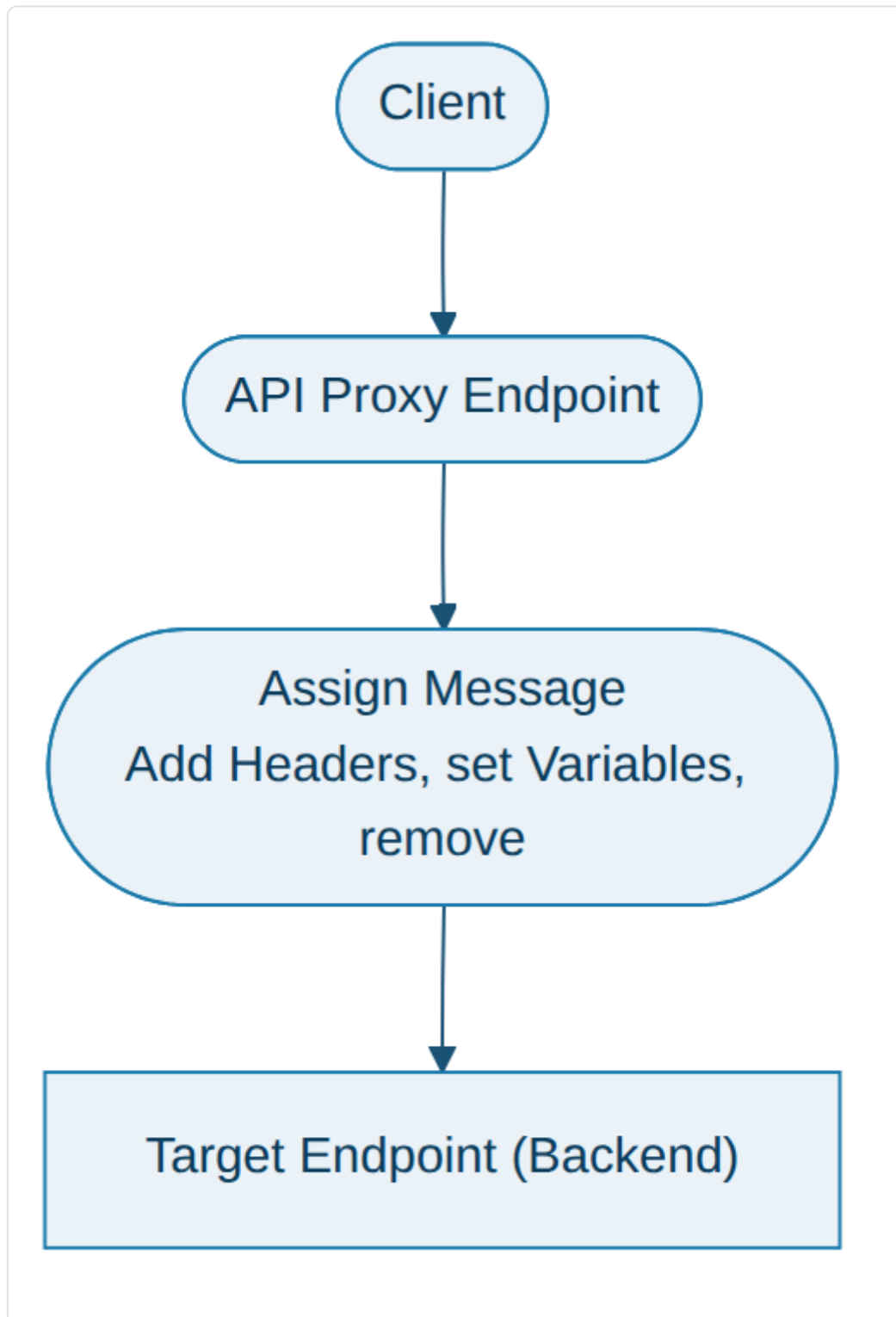
Jenkins Detailed Flow:



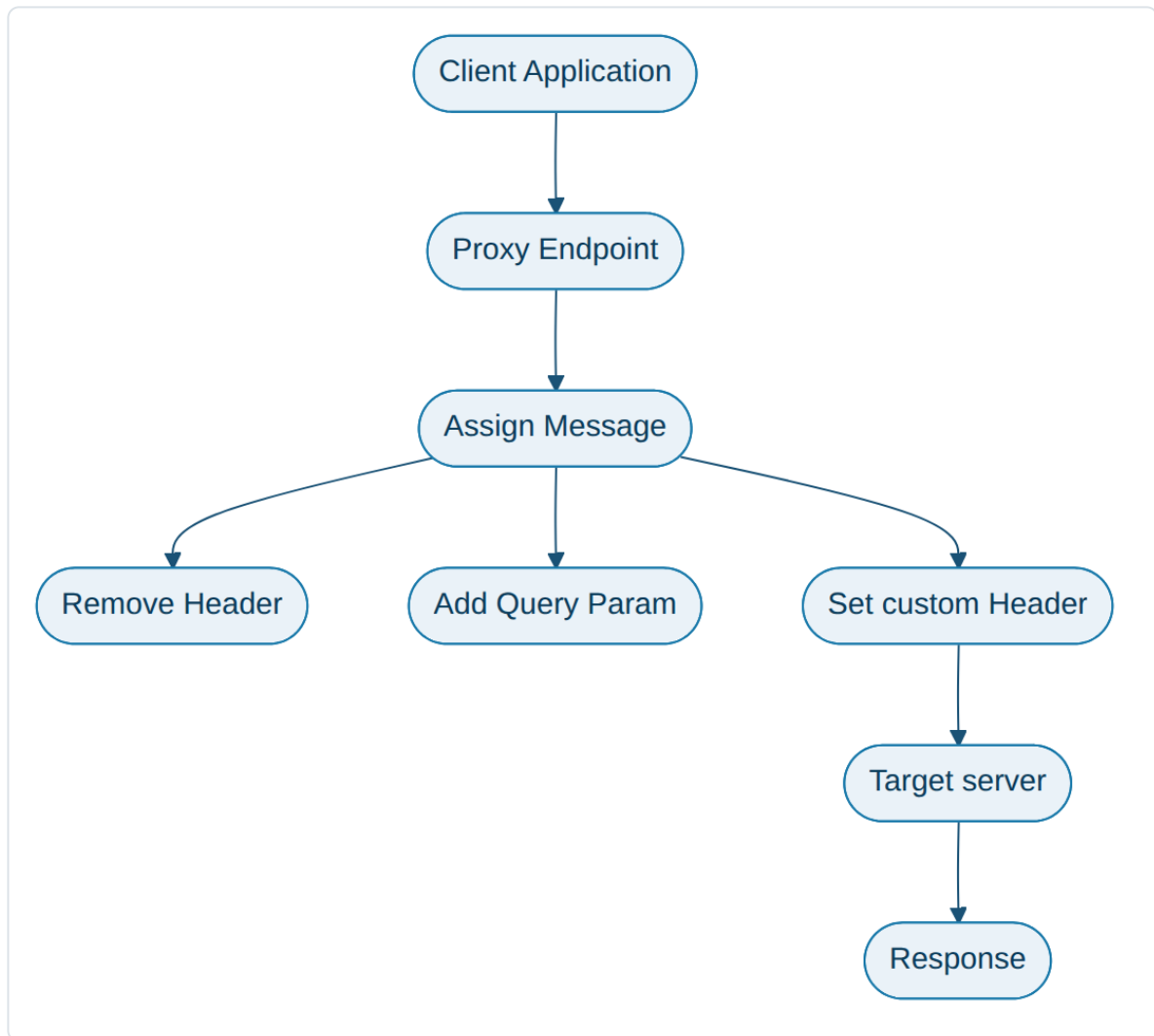
- For complex workflows we use Jenkins Pipeline (CI/CD).
 - Better for larger enterprises.
-

Assign Message policy

- Assign Message policy in Apigee is used to create, modify, copy, remove or set values in HTTP requests and response.
- Add or modify HTTP headers.
- Set query parameters.
- Create new requests.
- Modify Payload Content.
- Set Variables
- Remove Unwanted headers / parameters
- Transform requests before sending to the backend.



Detailed Flow Diagram:



Extract Variable policy

- Extract Variables policy in Apigee is used to extract data from incoming requests, responses, headers, query parameters, URI paths, payloads, or JSON/XML Content and store them into flow variables.
- These extracted variables can then be used by other policies such as
 - Assign message
 - Raise Fault
 - JavaScript
 - Service callout
 - OAuth policies
 - Route Rules.

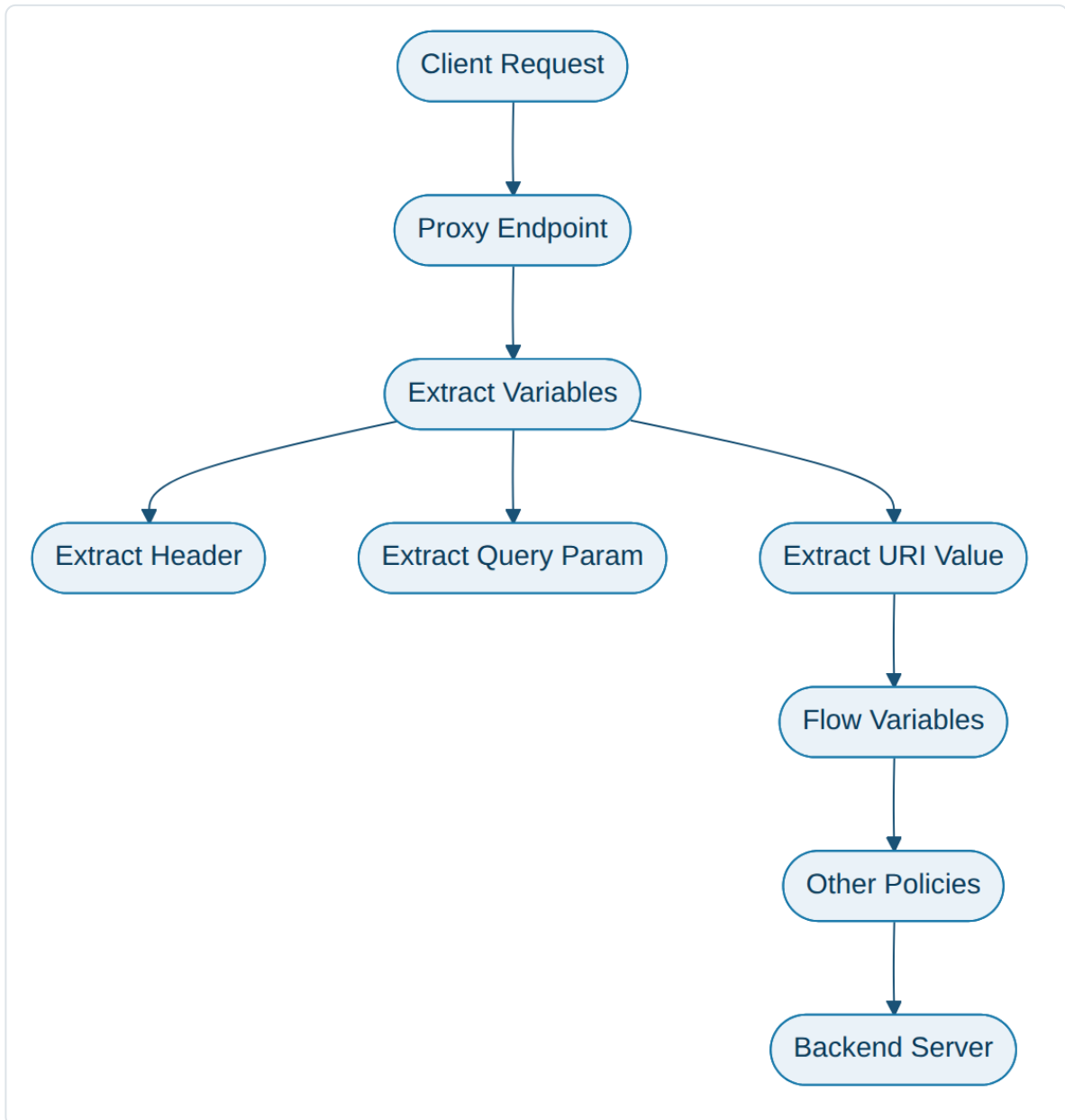
Eg

GET → /customers/123456 ? channel=mobile.

- Extract
- Customer ID = 123456

- Channel = mobile
- Store it in a flow variable & use it whenever required.

Detailed Flow Diagram:



Service callout policy

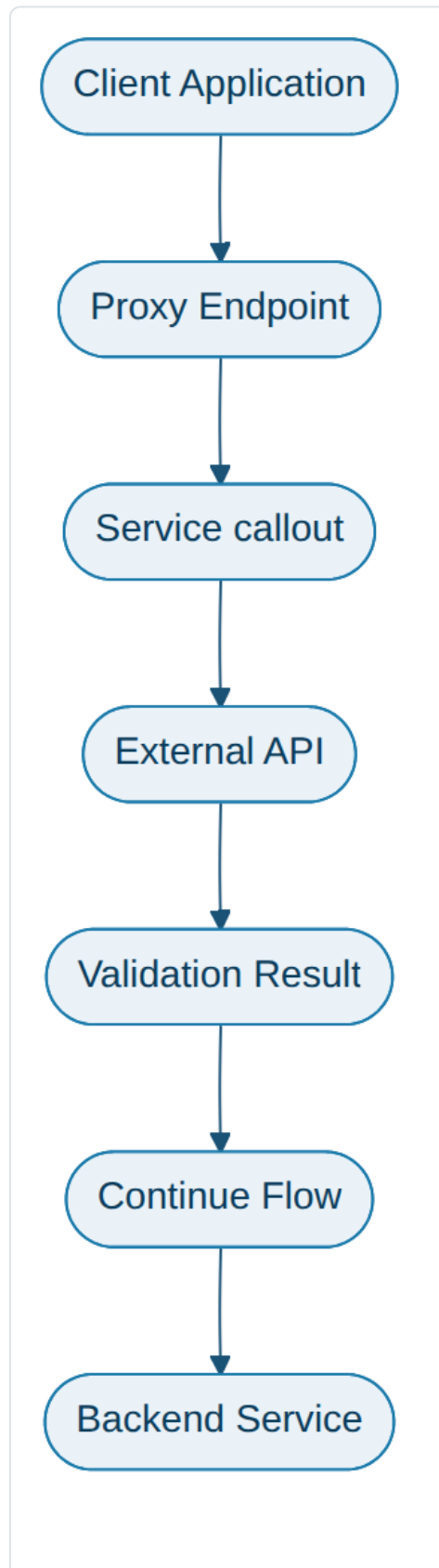
- Service callout policy is used to call an external service or API during the execution of an API proxy flow and use the response for further processing.
- One API calling another API from within Apigee before continuing the request flow.
- It's used for
 - Token Validation
 - Customer information lookup

- User profile retrieval
- External Authentication
- Calling internal microservices.

GET → /accounts/1001

- Before forwarding request to backend, Apigee
- Call a customer Validation Service.
- Verify whether account 1001 is active.
- Continue only if Validation succeeds.

Detailed Flow Diagram:



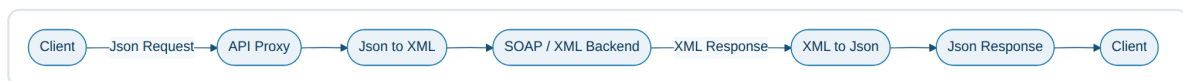
JSON to XML & XML to JSON policy

- JSON to XML policy is used to convert a JSON payload into XML format during API processing.

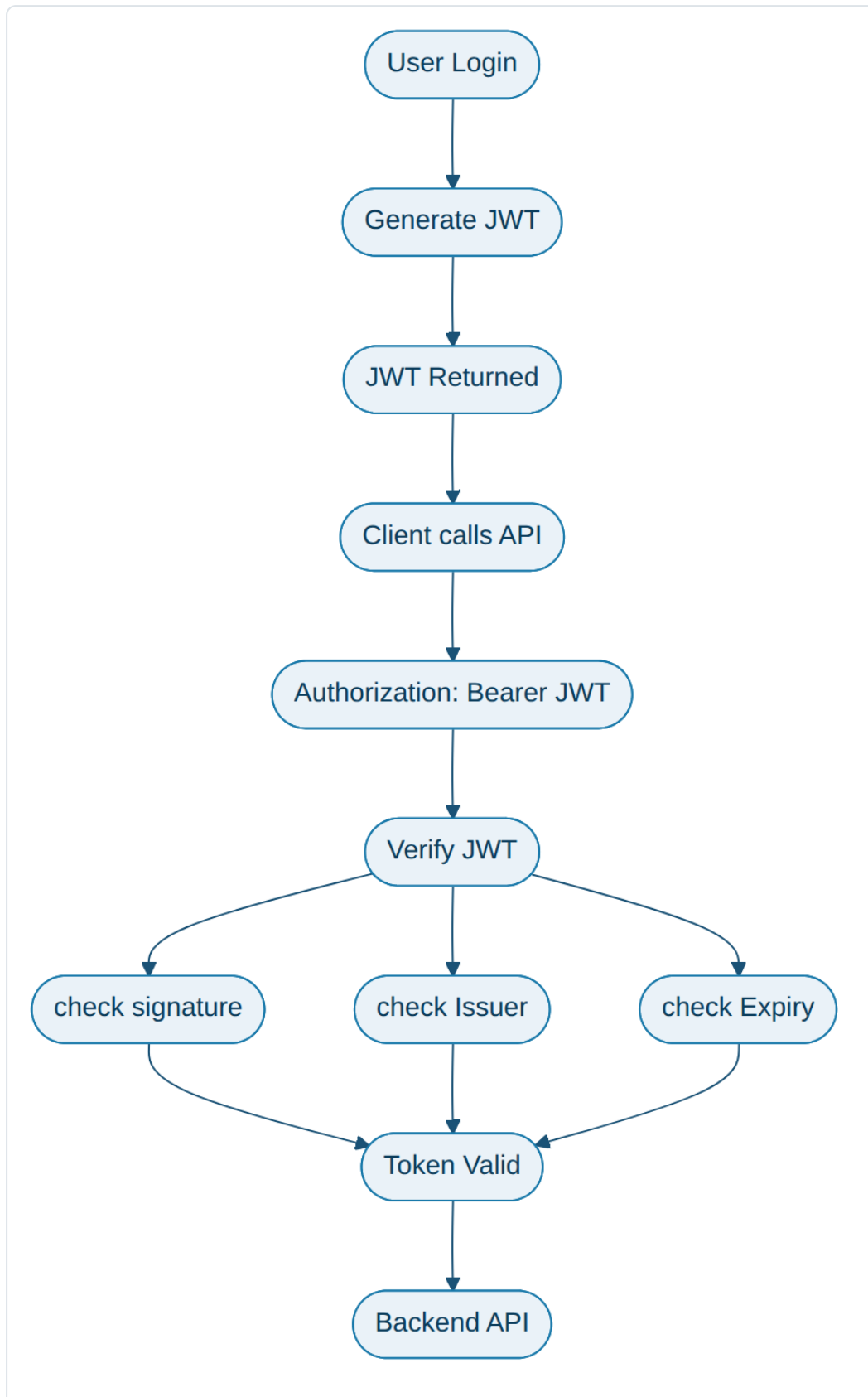
Eg

- Client sends JSON but backend accepts XML.
- SOAP backend integration, legacy systems require XML.
 - XML to JSON policy is used to convert XML payloads into JSON format.
 - SOAP backend returns XML
 - Client expects JSON.
 - Modern applications consume REST API's.
 - API modernization projects.

Detailed Flow Diagram



Flow



JSON & XML Threat Protection

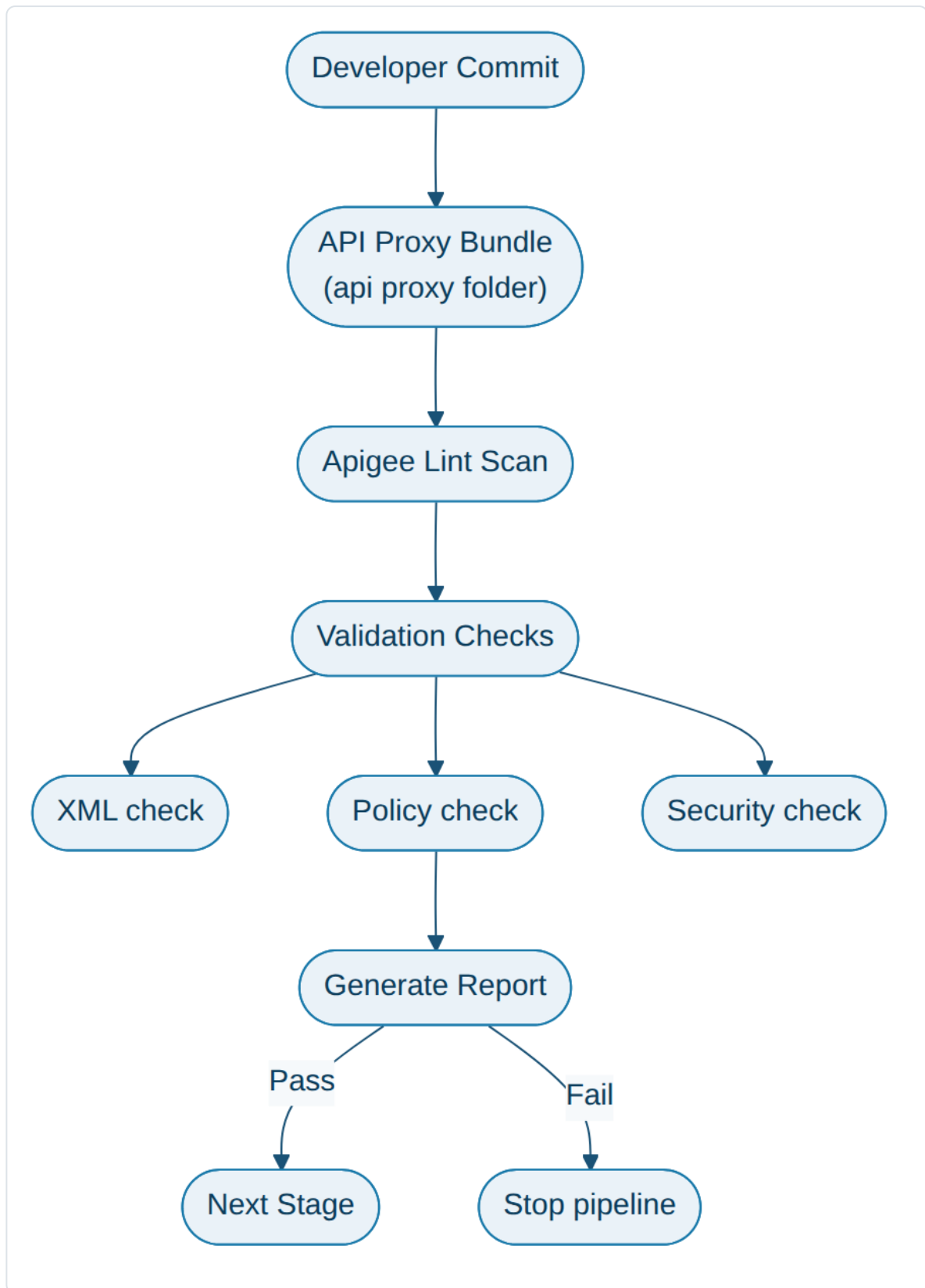
- These policies are security policies used to protect backend services from malicious (or oversized) JSON/XML payloads.
 - They help prevent attacks.
 - Denial of Service (DoS).
 - XML Bomb Attacks.
 - Deeply Nested Payload Attacks
 - Oversized Payload Attacks.
 - Malformed JSON/XML Attacks.
-

31/07/26

Apigee Lint Validation

- Validate API proxy structure, policies, naming conventions and best practices before deployment.

Flow Diagram



Steps

1. Install the official Apigee linter.

- `npm install -g apigeelint`
- `apigeelint --version`

2. Export the API proxy

- Download the proxy bundle from Apigee or use your source repository.
- Example structure.
 - SampleAPI
 - apiproxy
 - proxies
 - targets
 - policies
 - resources
 - SampleAPI.xml

3. Run Lint Validation

- `apigeelint -s SampleAPI/apiproxy`

4. Generate JSON Report.

- `apigeelint -s ./apiproxy -f json.js -e report.json`
- Output => `report.json`

5. Sample Console Report.

- Apigee Linter Report
 - Proxy: Customer API
 - Policies checked: 52
 - Passed: 47
 - Warnings: 3
 - Errors: 2

Eg → Error

- **ERROR P0006**
- Target Endpoint.xml
- → Target endpoint "backend-service" does not exist.
- **ERROR ST001**
- Proxy Endpoint /default.xml
- → Step "Verify API Key" references a missing policy.
- **WARNING BN001**
- Policy name
- → AssignMessage1 should follow naming convention.

- **WARNING P0013**
- → **AssignMessage.xml**
- Policy description missing.
- **WARNING FC001**
- → Default Fault Rule not configured.

Sample JSON Report

```
{
  "proxy": "Customer API",
  "summary": {
    "errors": 2,
    "warnings": 3,
    "passed": 47
  },
  "Issues": [
    {
      "severity": "error",
      "rule": "P0006",
      "file": "targets/default.xml",
      "message": "Referenced Target Endpoint does not exist"
    },
    {
      "severity": "Warning",
      "rule": "P0013",
      "file": "Policies/AssignMessage.xml",
      "message": "policy description missing"
    },
    {
      "severity": "warning",
      "rule": "BN001",
      "file": "policies",
      "message": "Policy name violates naming convention"
    }
  ]
}
```

CI/CD Integration Example: (YAML)

```
- name: Install apigeelint
  run: npm install -g apigeelint

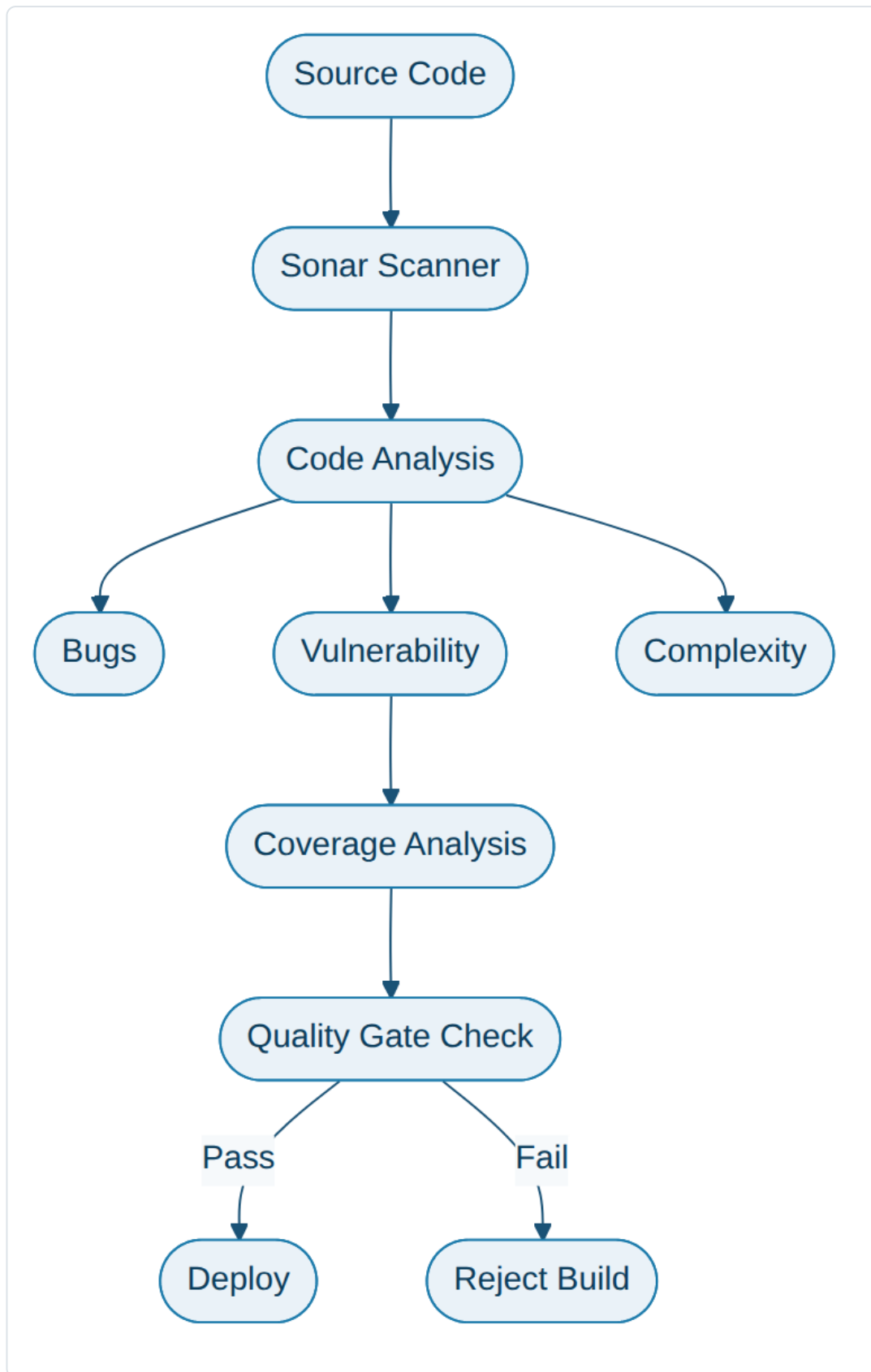
- name: Run Linter
  run: |
    apigeelint \
      -s apiproxy \
      -f json.js \
      -e lint-report.json

- name: Upload Report
  uses: actions/upload-artifact@v4
  with:
    name: apigee-lint-report
    path: lint-report.json
```

SonarQube Analysis Flow: (Quality std)

- Analyze JavaScript, Java Callouts, Node.js and Security Vulnerabilities.

Flow Diagram



SonarQube Quality Validation Steps

1. Install Sonar Scanner.
 - Download & Install the sonar scanner CLI.
 - Verify the installation.
 - `sonar-scanner --version`
 - Eg → O/P
 - SonarScanner 7.x.
 - Java 17.
 - Linux/Windows.
2. Configure SonarQube project.
 - Create a `sonar-project.properties` file in the project root.

Properties

```
sonar.projectKey = CustomerAPI
sonar.projectName = CustomerAPI
sonar.projectVersion = 1.0

sonar.sources = src
sonar.tests = test

sonar.language = Java

sonar.sourceEncoding = UTF-8

sonar.host.url = http://localhost:9000

sonar.token = xxxxxxxxxxxxxxxxxxxxx
```

1. Run Sonar Scanner
 - Execute the analysis.
 - `sonar-scanner`
 - (or specify the configuration explicitly.)

Bash

```
sonar-scanner \
-Dsonar.projectKey=CustomerAPI \
-Dsonar.sources=src \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.token=xxxxxxxxxxxxxxxxxx
```

1. Upload Analysis to SonarQube.
 - Scanner automatically uploads the analysis

Eg:

```
INFO Analysis report generated.  
INFO Analysis report compressed.  
INFO Analysis report uploaded.  
INFO  
INFO Analysis SUCCESSFULL.
```

1. View Results in SonarQube.

- Open your SonarQube server.
- Example dashboard sections
 - Overall Quality Gate
 - Bugs
 - Vulnerabilities
 - Security Hotspots
 - Code Smells
 - Coverage.
 - Duplicated Code
 - Maintainability
 - Reliability.
 - Security Rating.

2. Evaluate Quality Gate.

- Example Quality Gate.

Quality Gate

1. Status: PASSED.

Conditions

- Coverage $\geq$ 80%
- Duplicated Lines $<$ 3%
- Critical Bugs = 0.
- Blocker Issues = 0.
- Security Rating = A
- Maintainability = A

2. Quality Gate

- FAILED

- Coverage = 62%
- Critical Bugs = 3.
- Duplicated Lines = 8%

Sample Console Report

SonarQube Analysis Report

Project

Customer API

Files Analyzed

142

Bugs

Critical: 1

Major: 3

Minor: 5

Vulnerabilities

Critical: 0

Major: 2

Code Smells

Major: 16

Minor: 28

Coverage

82.16%

Duplicated Code

1.7%

Security Rating

A

Reliability Rating

A

Maintainability Rating

B

Quality Gate

PASSED

CI/CD → SonarQube Analysis (YAML)

```
image: maven:3.9.6-eclipse-temurin-17

definitions:
  caches:
    maven: ~/.m2
    sonar: ~/.sonar/cache

pipelines:
  branches:
    main:
      - step:
          name: sonarqube code Analysis
          caches:
            - maven
            - sonar
          script:
            # Build the Project
            - mvn clean verify

            # Run sonarqube analysis and wait for Quality Gate
            - mvn sonar:sonar \
              -Dsonar.projectKey=my-project \
              -Dsonar.host.url=$SONAR-HOST-URL \
              -Dsonar.token=$SONAR-TOKEN \
              -Dsonar.quality.gate.wait=true
```

01/08/26

ServiceNow → Apigee X (Hyperoptic Project)

- In our case → Infinity is a customer (or) consumer and Snow as a client.
- once we received the onboarding and KT from ServiceNow team
- 1st step → Before creating Api proxies in Apigee-X environment
- We will test the Snow Backend URL's. once these test gets succeed then only we will start the development.

Snow Authentication Mechanism

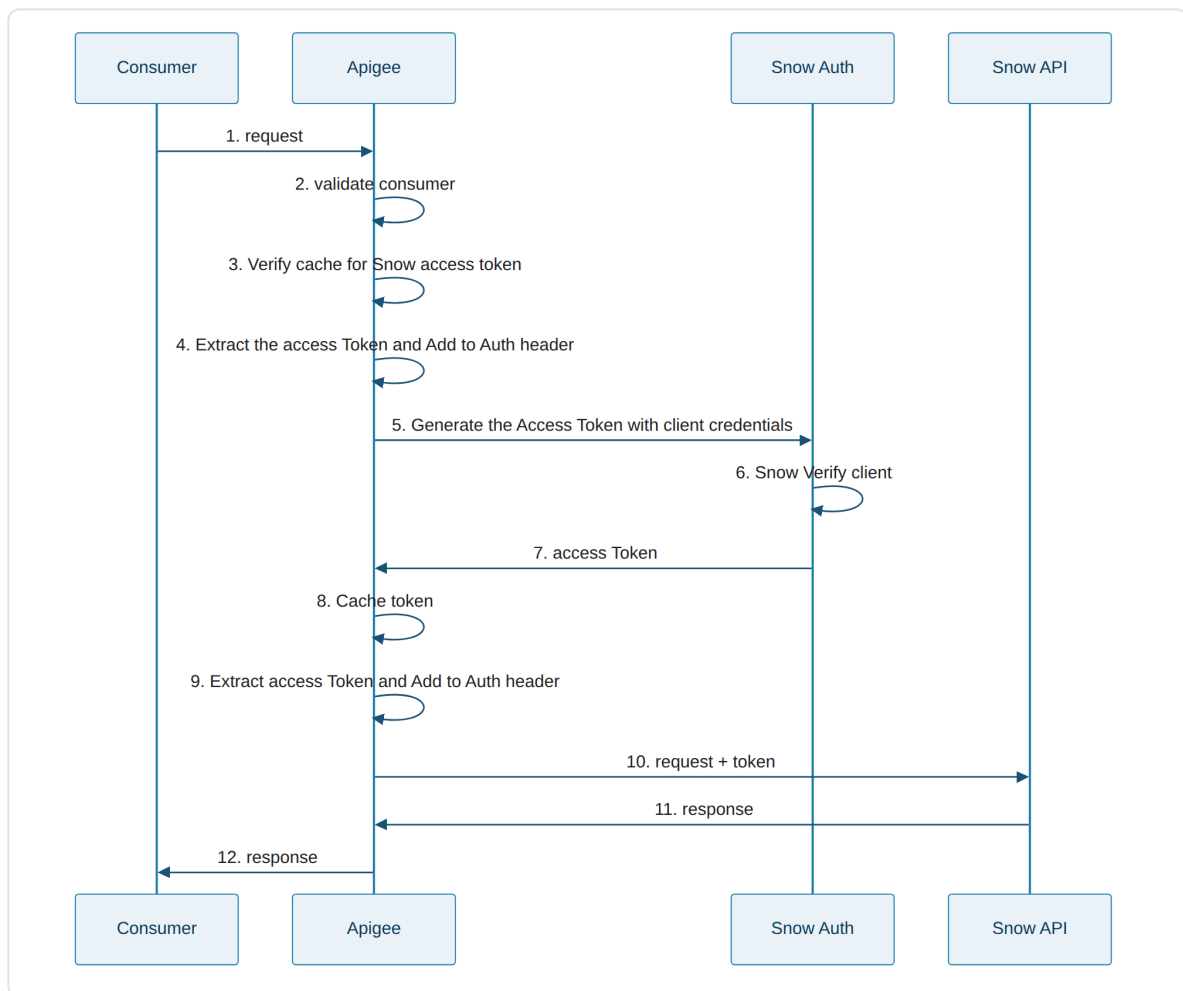
- In Snow Backend URL will not give the response without valid Snow access token.
- Snow Auth server → provide the access token → If credentials are Valid.
- once we have got the token, will use it in the request header as a Bearer.

Flow Diagram and steps

- Get the client credentials from Snow Team.
- 1st step → Use the client credentials to hit the OAuth URL & Get the token.
- OAuth URL
- client credentials → headers
- 2nd step → pass the Snow OAuth token in the request header of the Snow Backend URL.



Apigee X → Flow: (Snow)

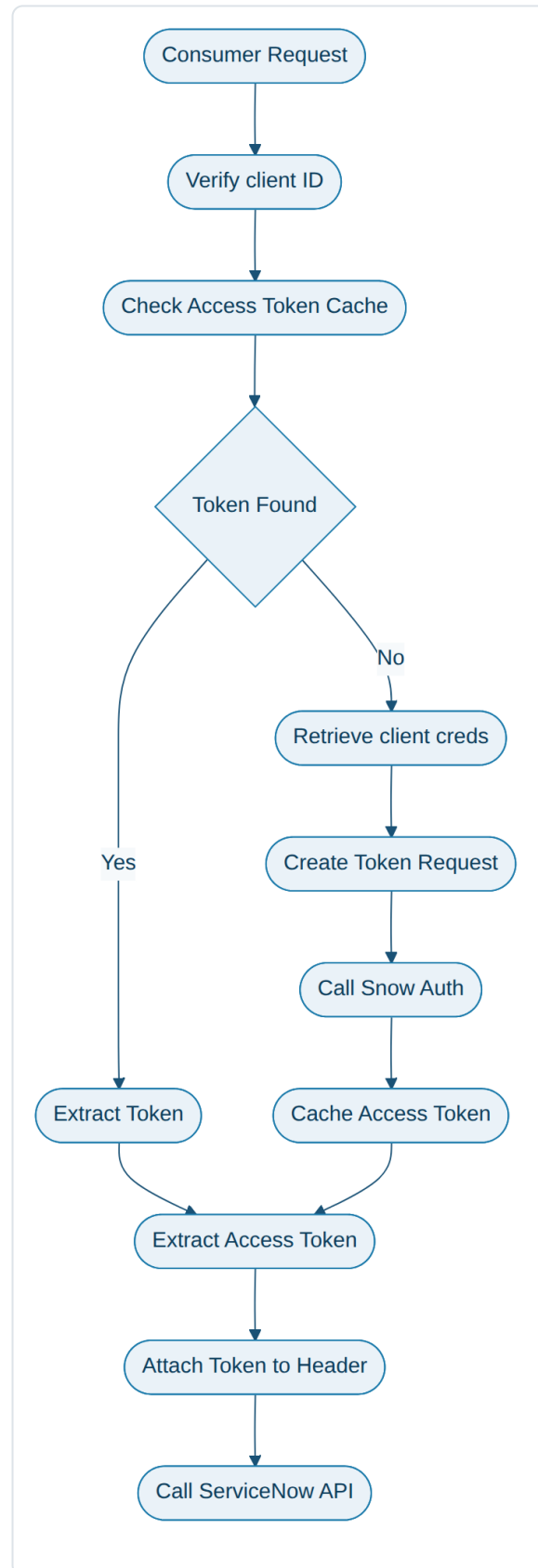


Flow Summary

1. Consumer sends a request to Apigee. (API Key)
2. Apigee validates the consumer

3. Apigee checks whether a ServiceNow access token exists in cache. (Available/not)
4. If token exists
 - Extract token from cache
 - Add token to Authorization header.
5. If token doesn't exist
 - Request a token from ServiceNow Auth using client credentials.
 - ServiceNow Auth verifies the client.
 - Access token is returned.
 - Apigee caches the token.
 - Apigee extracts token and adds it to Authorization header.
6. Apigee sends request with token to ServiceNow API
7. Snow API returns response.
8. Apigee forwards response back to the Consumer.

Apigee Policies used for this: (Snow)



Step-by-Step Explanation

1. (Verify API Key Policy)

- `sf-apigw-va-verify-client-id`
- Validate the API Key in the customer request.
- checks the incoming request contains a valid client ID.
 - Valid client → continue flow
 - Invalid client → Request Rejected.

2. (Lookup-cache policy)

- `sf-apigw-access-token-cache-lookup`
- check if a ServiceNow access token already exists in cache.
 - cache Hit → Use existing token
 - cache Miss → Generate a new Token.

3. `sf-retrieve-client-credentials` (KVM)

- Executed only when cache miss occurs.
- Retrieve OAuth credentials
 - client ID
 - client secret.

4. (Assign Message Policy)

- `sf-create-access-token-request`
- Build OAuth token request.
 - creates the request body for ServiceNow OAuth token generation.

Eg:

```
POST /oauth_token.do
- grant_type = client_credentials
- client_id = xxxxxxxxxx
- client_secret = yyyyyyyy
```

1. (Service callout policy)

- `sf-apigw-sc-request-to-snow`
- Call ServiceNow Authorization server
- Sends the OAuth request.
- ServiceNow Validates the credentials.
- ServiceNow returns the Json response (Access Token).

2. (cache policy - populate)

- `sf-apigw-cache-access-token`
- store the token for reuse
- saves the newly generated token in Apigee cache.

3. (Assign Message Policy)

- `sf-apigw-extract-access-token`
 - Extract the access token value.
 - caught the token path.

4. (Assign message policy)

- `sf-apigw-attach-access-token`
 - Add the token to the outgoing request.
 - creates the Authorization header.

Eg → `Authorization: Bearer {Token}`

Infinity → Apigee X (Hyperoptic Project)

- We build the API proxy for Infinity client.
- when ServiceNow as a customer & infinity as a client.
- once we received the onboarding sheet from the infinity client and before creating the Apigee API Proxy we will do the testing at our end on Infinity backend API's.
- once the testing is get succeed after that we will implement the API proxies in Apigee X.

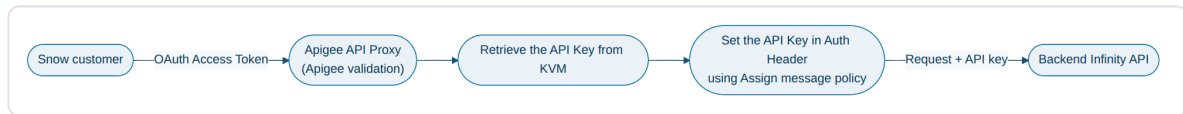
Infinity Auth Mechanism

- Infinity Backend is secured by an API Key.
- when we are testing the Infinity backend, we need to pass the API key in the Authorization header.
- once these key is valid then request will continue else will get the 401 Error (Invalid API key).

Apigee Flow

- In Apigee X → For Infinity client → ServiceNow as a customer.
- We implemented OAuth 2.0 Access token mechanism to the Snow customers (Infinity client).
- Apigee will validate the access token which snow provides the Auth header (Bearer Token)
- once token is valid the request is send to the backend.
- For Infinity Backend we need to authenticated by Infinity API Key.
- After OAuth Validation by Snow → Apigee next flow is to retrieve the API Key (Infinity one) from KVM (Key Value Map).
- After retrieved the value from KVM → we need to set the value in Auth header to forward the request to its Infinity Backend.
- Using Assign message policy to set the header value before making the request to backend.
- Without API key (Infinity) Backend will not give the response.

Flow Diagram → Infinity to Apigee



11/8/26

L&T → Interview

→ AWS → Apigee Gateway

JD:

Apigee	AWS
API proxy creation	EC2
API Product	S3 (storage)
Developer Apps.	VPC
Developers	Route 53
Target Servers	RDS
KVM (Key Value Maps)	Lambda
Shared Flow	Cloud Watch
Flow Hooks	Dynamo DB
Policies	IAM
API Analytics	WAF
Swagger	Cloud Front
	SNS
	SQS
	SES

Apigee Important Policies

1. Security Policies

- OAuth v2

- Verify API key
- JWT Generate/Verify
- Spike Arrest
- Quota

2. Traffic Management

- Spike Arrest
- Quota
- Response Cache

3. Transformation

- Assign message
- Extract Variables
- JavaScript Policy
- Message logging

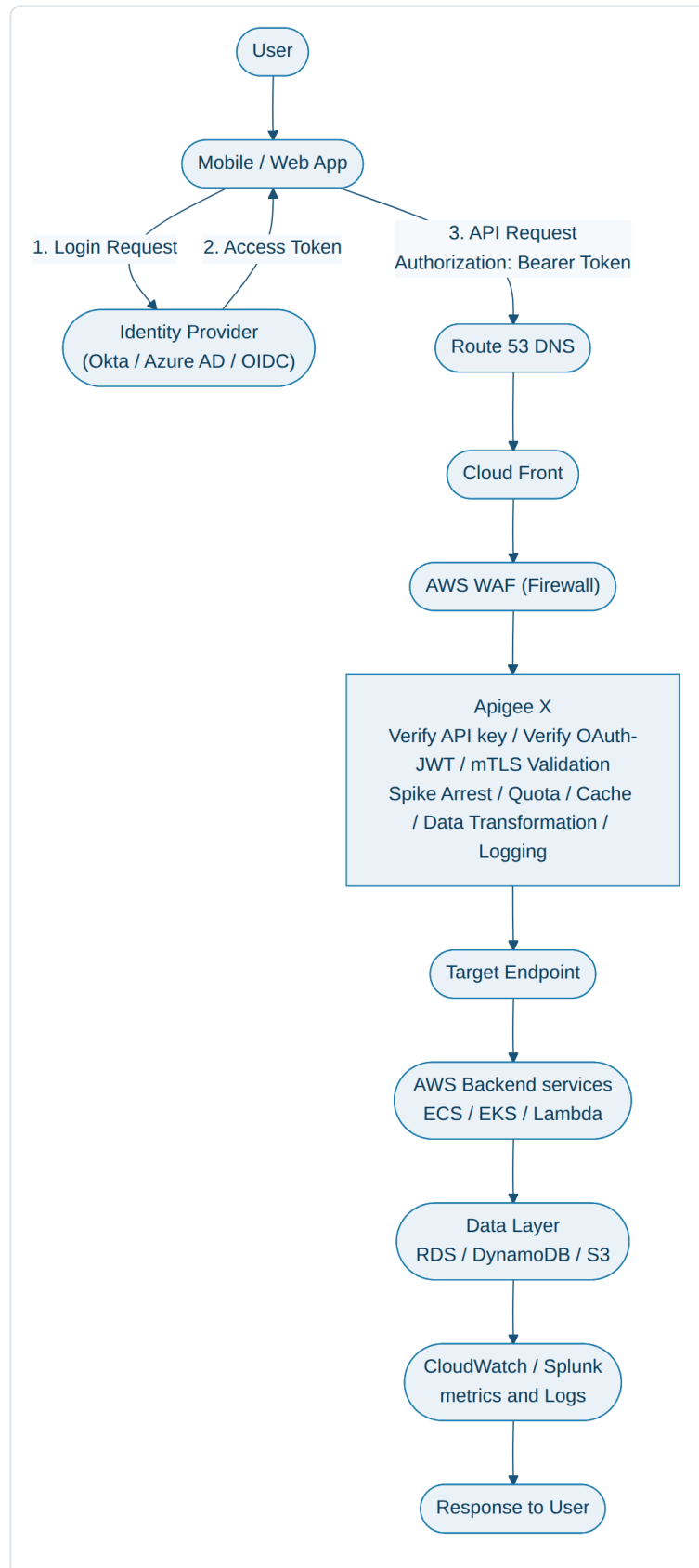
4. Error Handling

- Raise Fault
- Fault Rules

Security Topics

- OAuth
- mTLS/TLS
- Authentication
- Authorization
- Certificates
- HTTP status code
- Open IDC (OIDC) → Identity Provider
- Synchronous & Asynchronous → Apigee
- Production support
- SLA Tickets
- Troubleshooting
- What is the difference b/n AWS/Azure/GCP Gateway (what are the benefits)
- Yamuna → Need to learn → how to talk smartly → Interviewers (manage)

Flow Diagram: (AWS → Apigee Gateway)



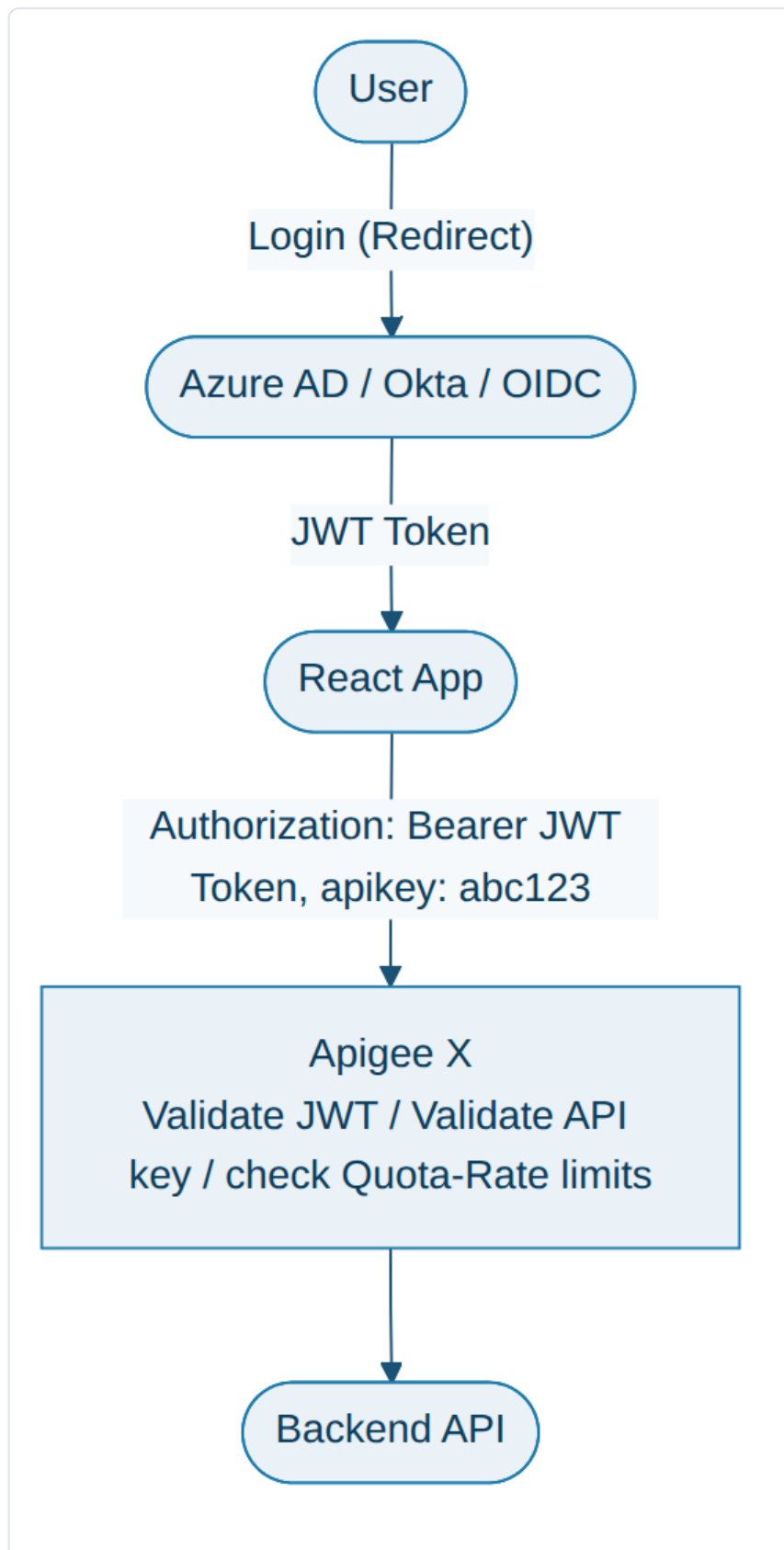
Steps

1. User logs into the application
 2. Receives an OAuth access token from Identity provider (Okta/Azure AD/OIDC)
 3. User invokes the API's through Route 53, Cloud Front and WAF
 4. Apigee acts as the API Gateway and performs
 - OAuth Validation
 - mTLS verification
 - Throttling through Spike Arrest
 - Quota policies
 - request transformation
 - routing
 5. Request is forwarded to the backend services hosted on
 - ECS
 - EKS
 - Lambda
 6. This calls data from RDS, DynamoDB or S3
 7. Logs and Metrics are sent to Cloud Watch and Splunk for monitoring before the response is returned to the user
-

12/8/26

Full-stack Application - Apigee

Flow Diagram: (Microservices)



Steps

1. User opens the React application
 - User accesses the frontend URL
2. User clicks login
 - React redirects the user to the Identity provider
 - Azure AD, Okta, AuthO, OIDC (Open ID Connect)
3. User authenticates
 - Azure AD displays the login page
 - Username
 - Password
 - MFA
 - User enters credentials
 - Azure AD verifies the user identity
 - Authentication successful
4. Azure AD returns Authorization Code
 - After successful login, Azure AD redirects the browser back to React
 - React now has Authorization Code but not the access token yet
5. React Exchanges Code for Access Token
 - React sends the authorization code to Azure AD Token Endpoint
 - Azure AD validates the Authorization Code and generates the JWT Token
 - Now React receives the OAuth Access Token (JWT)
6. React stores the Access Token
 - React stores the Access Token
7. React calls Apigee API
 - React sends API request with the access token (Apigee URL)
8. Request reaches Apigee
 - Apigee receives

```
GET /users/123
Authorization: Bearer eyJhbGciOi.....
```

- Before routing, Apigee executes security policies
9. Apigee validates JWT/OAuth token
- Using **Verify JWT** Policy
 - Apigee checks
 - Signature
 - Issuer
 - Audience
 - Expiration Time

- OAuth scopes
- User claims
- If Validation fails
 - 401 → Unauthorized
- and Request stops here
- 10. Apigee applies Additional policies
 - Apigee may additionally perform
 - API Key validation
 - Rate limiting
 - Spike Arrest
 - Quota Validation
 - Threat protection
 - Logging
 - Analytics
- 11. Apigee Routes Request to Backend Microservice
 - Apigee forwards request to the configured target endpoint
 - Request forwarded
 - Backend process the request and fetches data from the database
- 12. Backend Response Returns to User
 - Backend Responds (below flow)

Flow



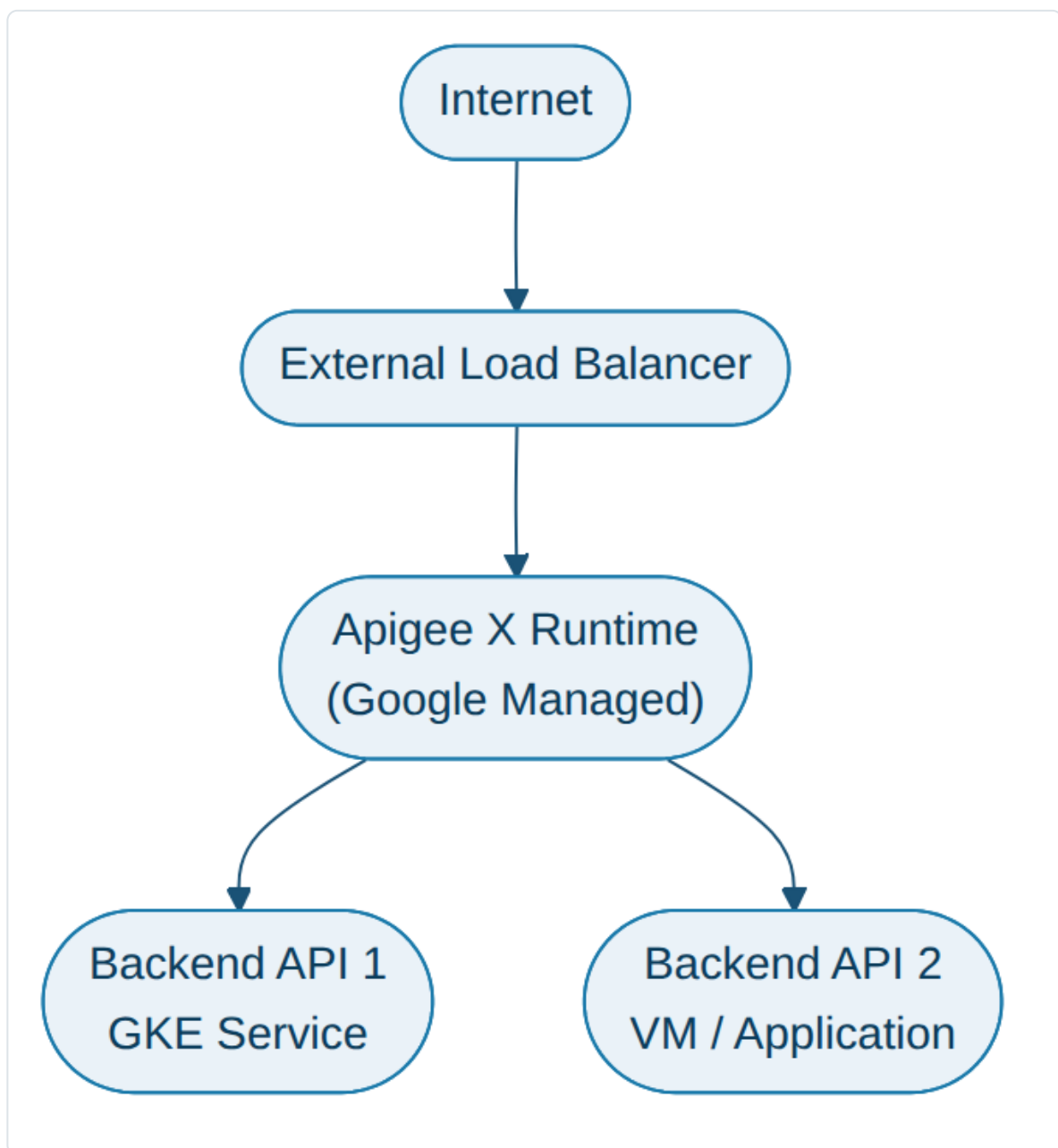
14/8/26

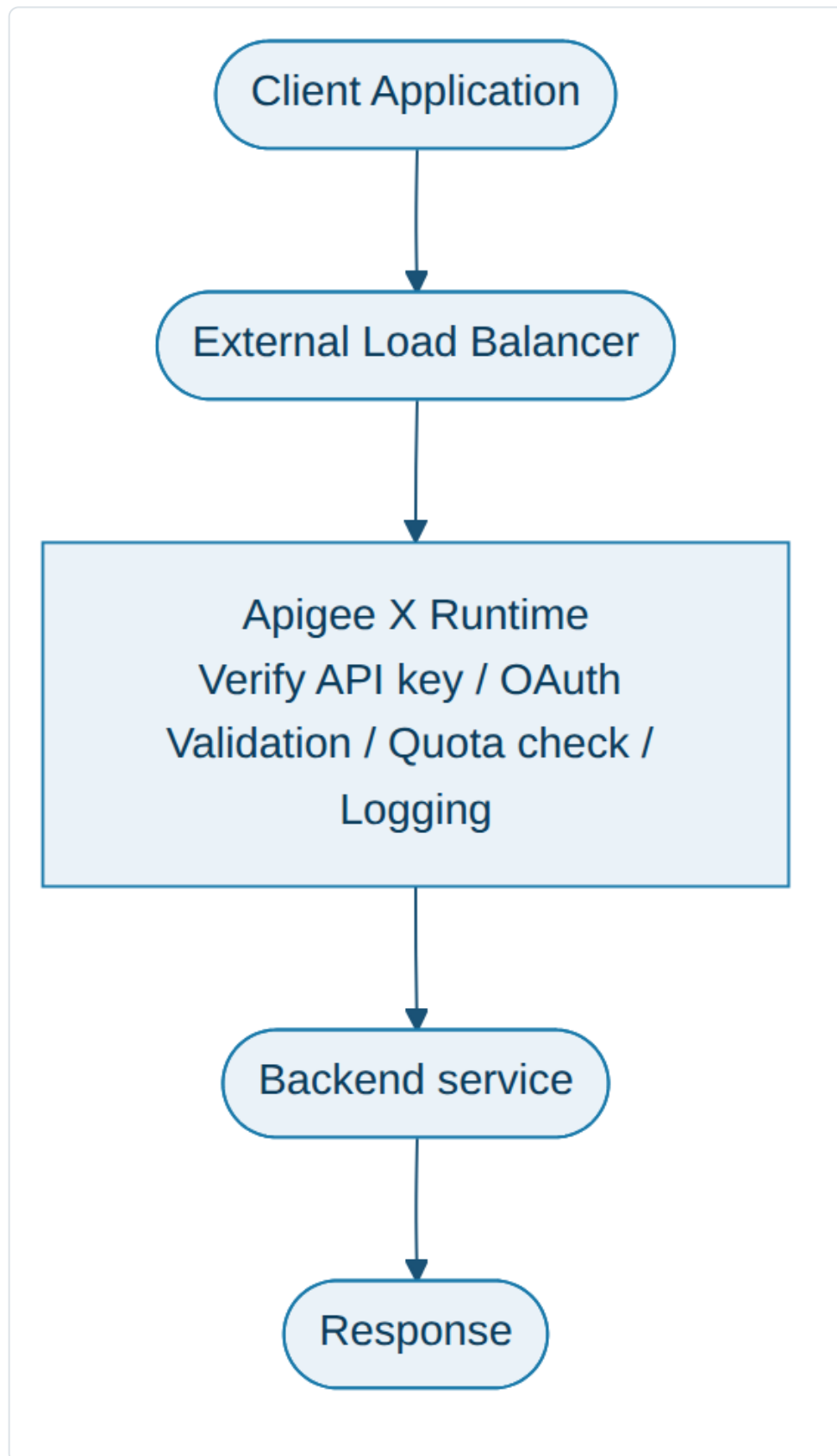
Apigee X - GCP

- Apigee X is Google's fully managed API Management platform
- As an API developer → you create
- API proxies
- Policies
- API Products
- Developer Apps

- Security rules
- Google manages
- Runtime Infrastructure
- Scaling
- High Availability
- Underlying Kubernetes
- Platform upgrades

Apigee X → Architecture Diagram





Step by step Explanation

1. client sends request
 - GET /customers/100.
2. Request reaches Apigee X Runtime.
3. Policies execute
 - VerifyAPIKey
 - OAuth V2
 - Quota
 - Spike Arrest
 - AssignMessage
4. Request is forwarded to backend.
5. Backend sends response
6. Apigee may transform response & return.

Important Apigee X components

1. Organization
 - Top level Apigee container
2. Environment
 - Deployment stage
 - Dev, Test, Prod.
3. Environment Group
 - Maps hostname to environments
 - eg → api.company.com
4. API proxy
 - Gateway between client and backend.

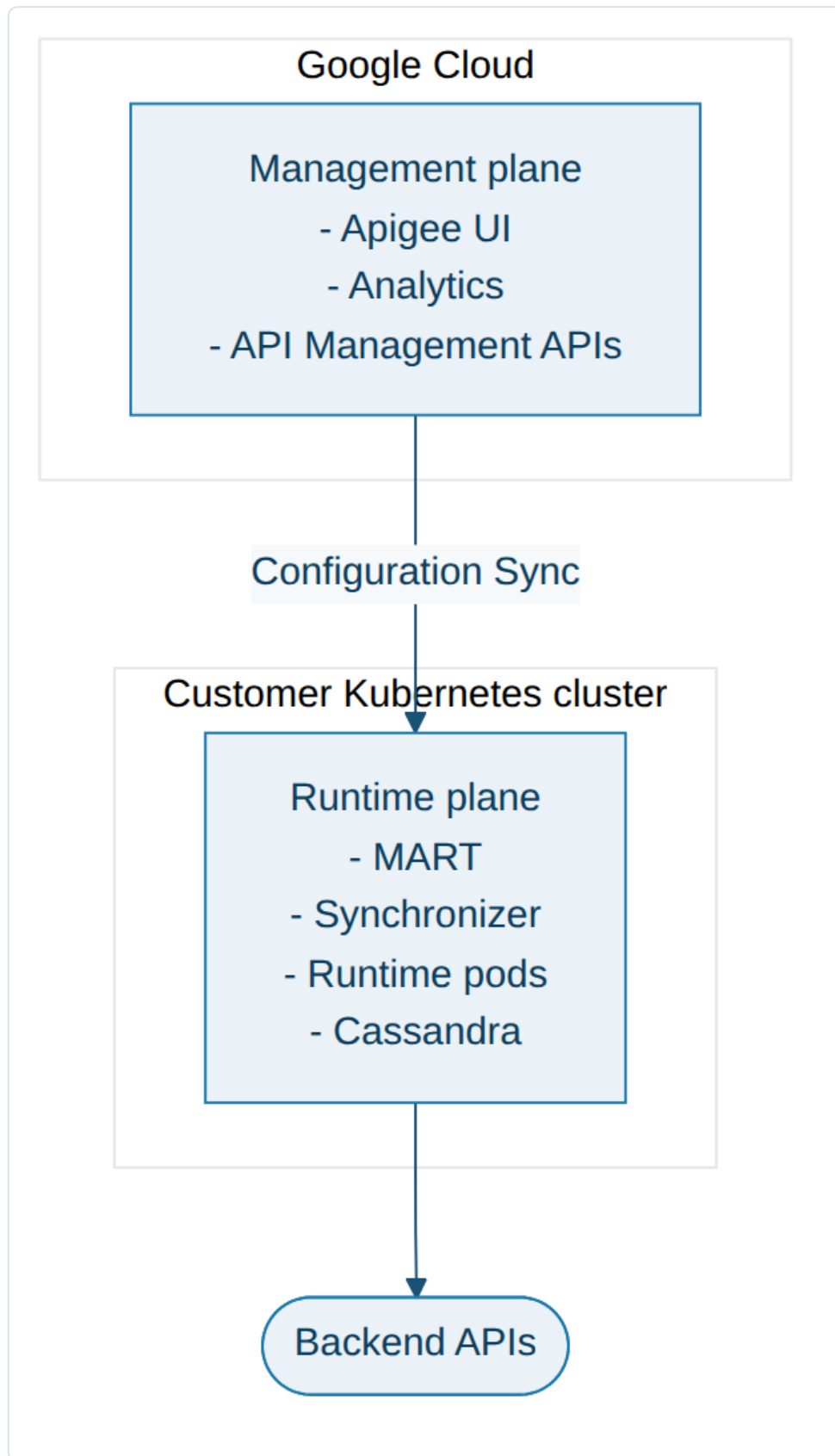


1. Analytics
 - Traffic
 - Latency
 - Errors
 - Usage
 - Success Rate

Apigee Hybrid → GCP + Onprem

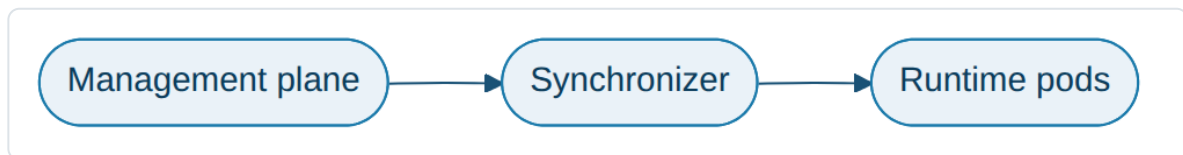
- Apigee Hybrid separates the platform into
- Management plane. (Managed by Google).
- Runtime plane. (Runs inside your Kubernetes cluster).
- Google documentation describes Hybrid
- cloud-hosted management services
- customer-managed Kubernetes runtime.
- All API traffic processed in the runtime plane.

Apigee Hybrid → Architecture Diagram



Runtime components in Hybrid

1. Runtime pods
 - These act as API Gateways.
 - Responsibilities
 - Authentication
 - Authorization
 - API routing
 - Transformation
 - Quota
 - Logging.
2. Synchronizer
 - Downloads configurations from Google management plane.
 - Examples
 - API proxies.
 - Shared Flows.
 - Environment Configuration

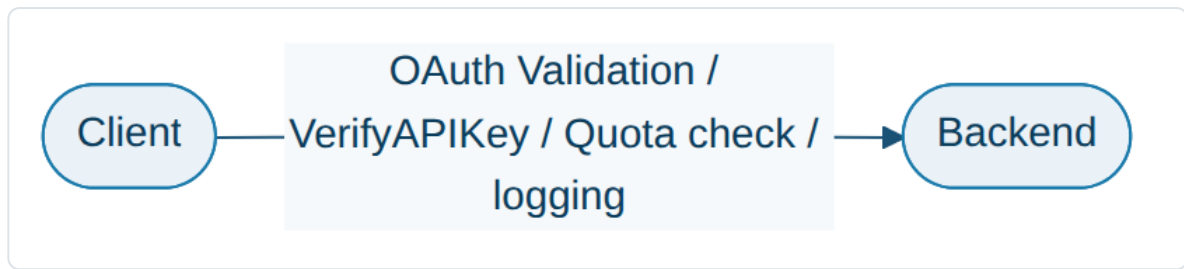


1. MART
 - Stands for Management API for Runtime.
 - Provides runtime management communication.
2. Cassandra
 - Stores runtime data
 - OAuth Tokens.
 - KVM Data.
 - Caches.

API Traffic Flow in Hybrid



Request processing

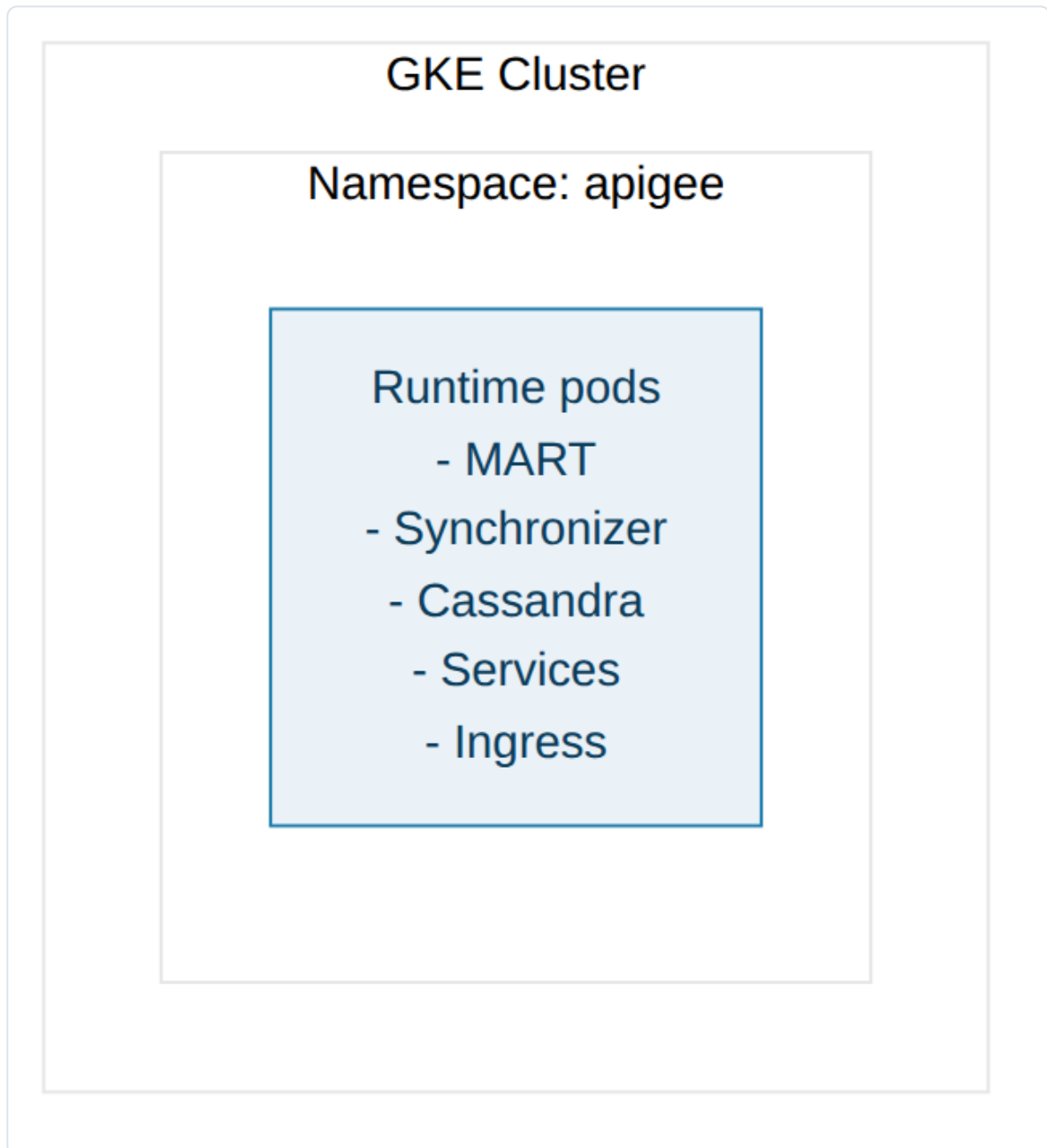


- API traffic never goes to the Google Management plane.
- Traffic stays within your runtime environment.

How Kubernetes Fits into Hybrid

- In Hybrid → Apigee is installed inside Kubernetes.

Eg:



- Kubernetes Manages
- Pod scheduling
- Scaling
- Self-healing
- Load distribution.

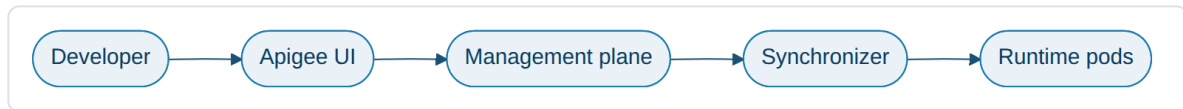
Comparison b/n Apigee X & Apigee Hybrid

1. Apigee X
 - Google manages both the Control Plane and runtime plane.

2. Apigee Hybrid

- Google manages → control plane
- Customer manages the runtime plane on Kubernetes.
- All API traffic is processed inside the customer-managed runtime environment.

Configuration Flow



Steps

1. Create an API proxy

- You create API proxy (customer API) in the Apigee UI
- Eg → GET /customers
- also add policies
 - VerifyAPIKey
 - Quota
 - OAuth

2. Stored in Management Plane.

- Proxy definition is stored in Google's management plane.
- Think of it as
 - Google cloud
 - Management plane
 - API proxies
 - Shared Flows
 - products
 - Developers
 - Configuration

→ At this point

- Traffic is NOT using it yet
- runtime doesn't know about API yet

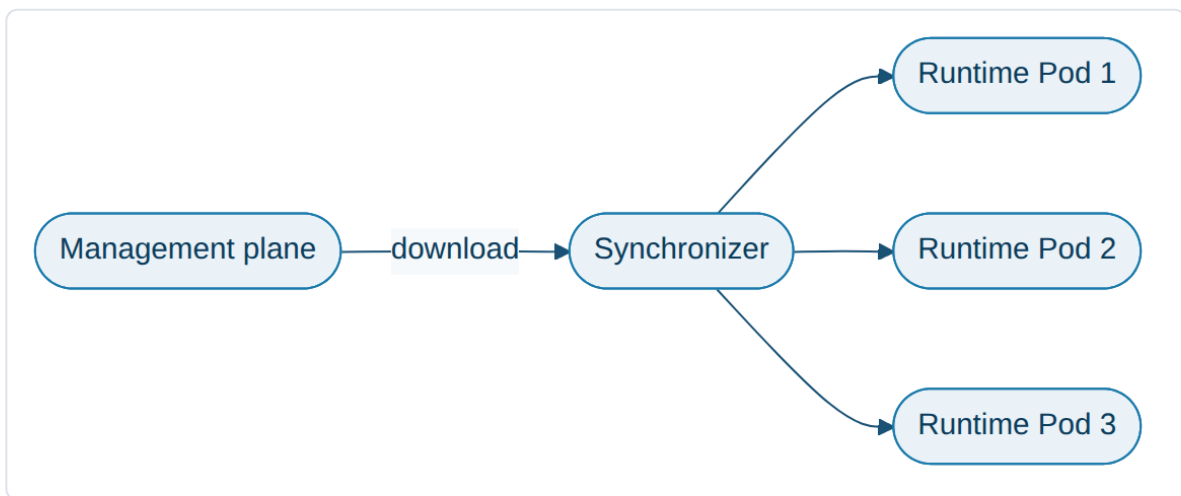
1. Synchronizer checks for changes.

- Synchronizer runs inside your Kubernetes cluster.
- Eg →
 - Kubernetes
 - synchronizer pod.
 - Its job is simple.
 - Ask Google if there are any new configurations.

- Think of it like → Git Pull for Apigee.

2. Synchronizer Downloads config.

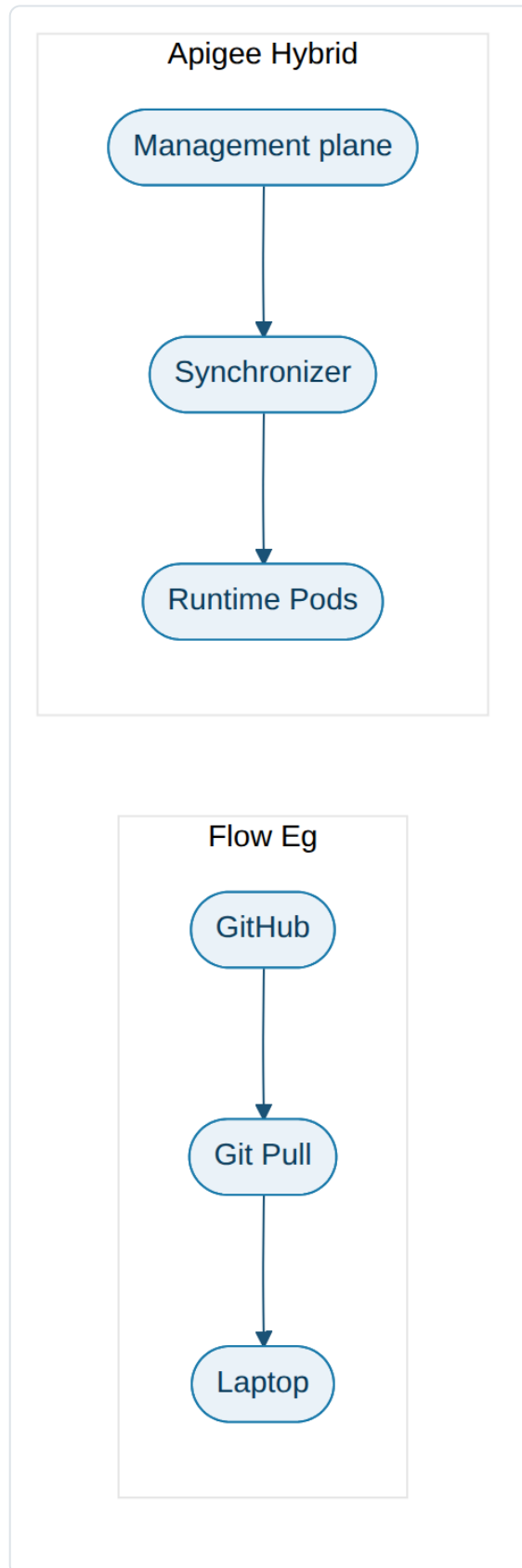
- Synchronizer fetches
 - Customer API proxy
 - OAuth policy
 - Quota Policy
 - Environment config
 - Shared Flows
- from the Management plane.



1. Synchronizer pushes to Runtime.

- Now Synchronizer updates the Runtime pods.
 - All Runtime pods receive the new configuration.
- Now they know
- /customer API exists.
 - VerifyAPIKey enabled.
 - Quota enabled.

Flow Eg



What Happens if Synchronizer Stops

- Suppose synchronizer crashes.
- Synchronizer = Down.
- Existing API's → still work
- Customer API
- Order API
- Payment API
- Continue serving traffic (Existing runtime configurations remain loaded).
- New Deployment → Fail to sync.
- New API proxy
- New Shared Flow
- New Quota Policy
 - Won't reach Runtime pods until synchronizer is working again.